

Information Retrieval and Text Mining

Detecting Patterns and Organizing Text – Part 1

Prof dr ir Jan Scholtes
Edition 2024-2025



Maastricht University

Learning Goals

- Understand different supervised methods for document classification such as Naïve Bayes, Decision Trees, Logistic Regression, k-NN and Support Vector Machines (SVM)
- Understand Deep-Learning approach towards text classification
- Understand sentiment & emotion mining
- Understand the detection of toxic texts

Text Classification



Maastricht University

Why is Text Classification Relevant?

Text classification is no longer just about "sorting" text; it is the decision-making engine for modern AI systems. By transforming unstructured text into structured categories, we enable machines to understand intent, prioritize risk, and orchestrate complex tasks such as:

- Agentic intent discovery and workflow routing.
- Structuring unstructured text for Retrieval Augmented Generation (RAG).
- Safety, compliance and moderation (detect and address toxic language).
- Reducing the “linguistic noise”: Text classification addresses the "Long Tail" of natural language. By categorizing ambiguous or slang-heavy text into standardized labels, it creates a reliable "Target Representation" that statistical and neural models can process efficiently at scale.

In this lecture we will address several aspects of supervised text classification. In the next lecture we will look more into unsupervised methods.

Decision Trees

Entropy modeling
Random Forrest



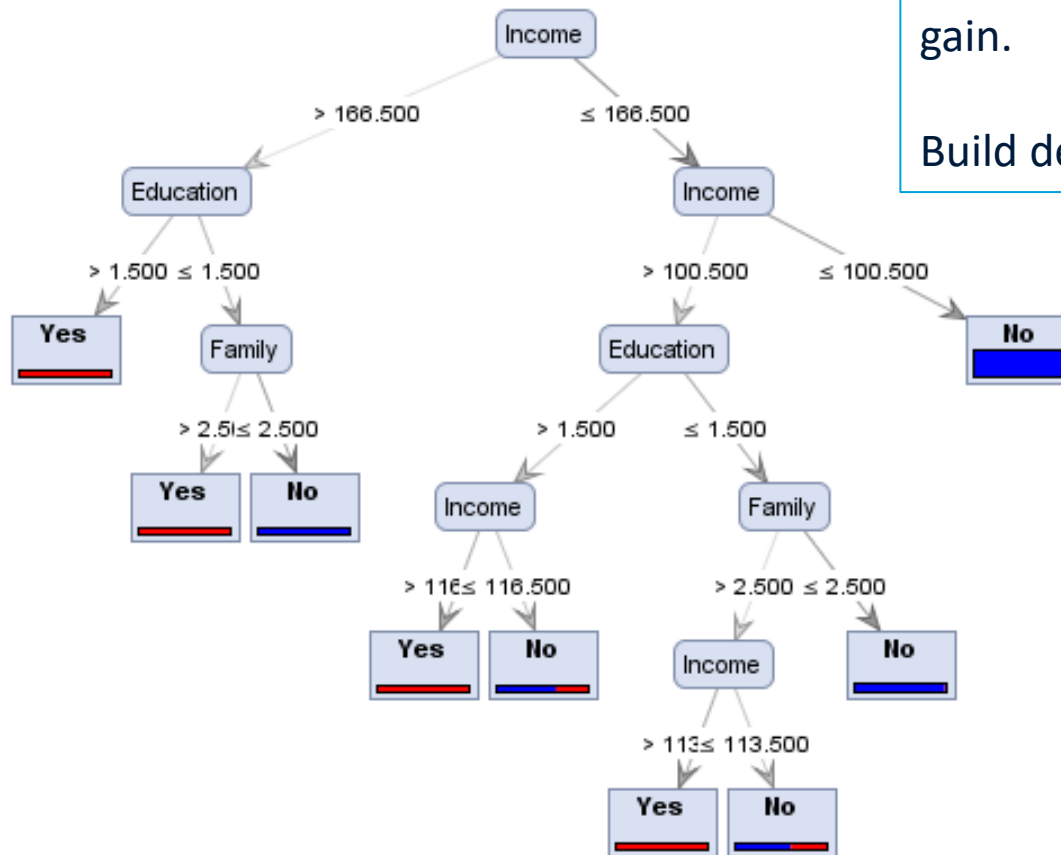
Maastricht University

Decision Trees & Entropy Modeling

Good basic method for text-classification.

Use TF-IDF values to calculate information gain.

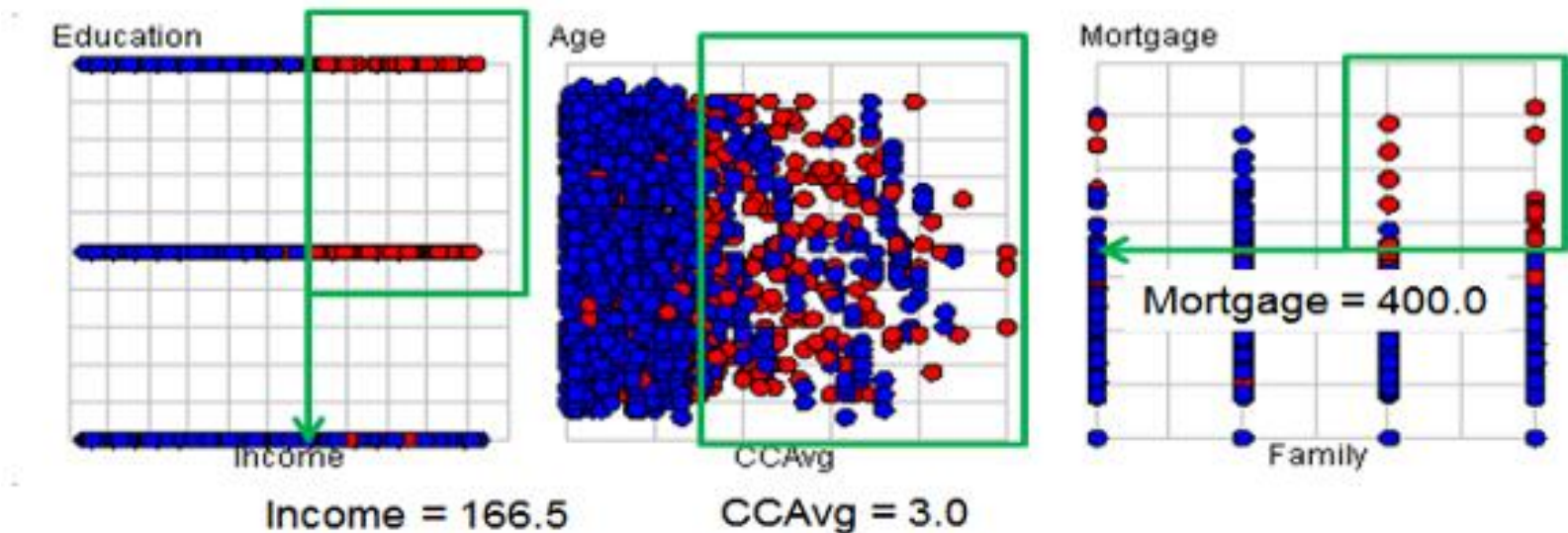
Build decision similar as shown before.



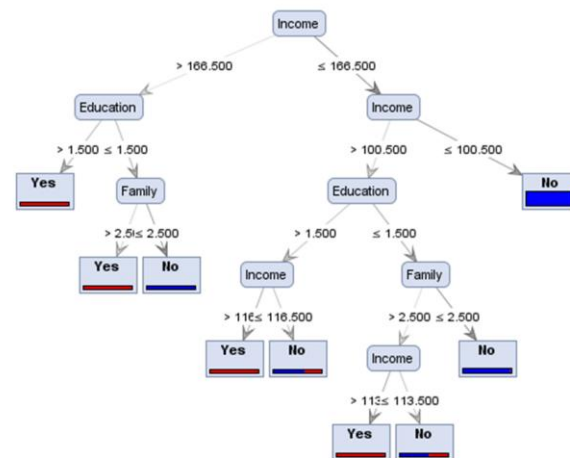
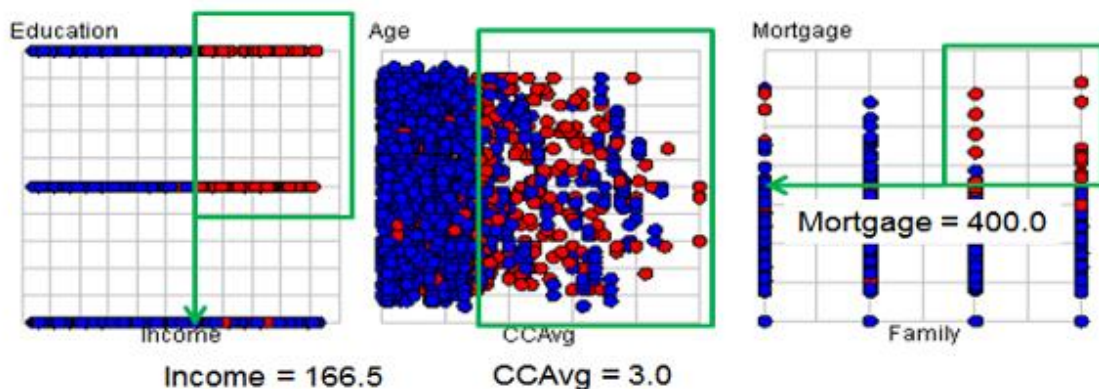
"how does the model know at what value to split the top node?"

"how does the model determine the order of the nodes from top to bottom?"

The core idea behind decision tree algorithms is that of dataset homogeneity. In the series of plots below, the **red** points indicate loan acceptance ("Yes") and **blue** indicate "No" or non-acceptance. From a visual inspection, we can see that if the data is split along the "Income" axis at approximately 166.5 (and education at 2) we can separate the dataset into two homogeneous groups of mostly all reds and mostly all blues. But one could perhaps make a similar (subjective) argument for Mortgage at 400.0 and Family at 3 or even CCAvg at 3.00.



For the data subset that is 100% homogeneous (all Yes or all No), the entropy of the target variable is zero and for a subset that is a perfect 50-50 mixture, entropy of the target is 1.0. In our dataset, the target variable entropy will lie somewhere in this range (0.0 to 1.0). If we compute the entropy for each of the above three subsets within the green rectangles, we will find that the subset with the lowest entropy is the green rectangle in the Income-Education plot.

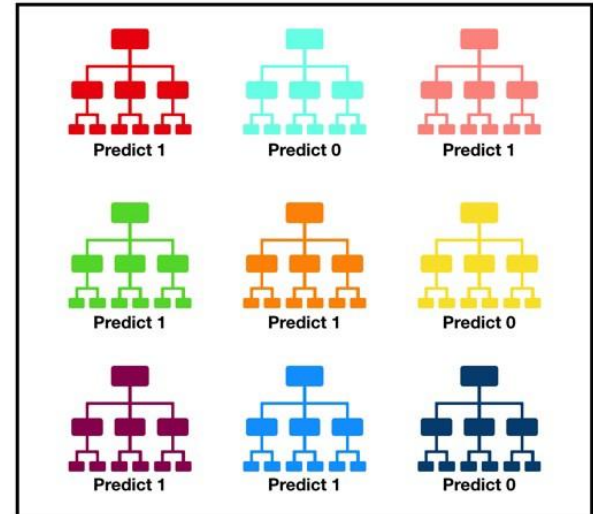


The attribute which **minimizes entropy (or maximizes information gain)** is selected as the **root node or top node**. Rank ordering the entropy will yield the other nodes or attributes where splitting the data resulted in decreasing entropy.

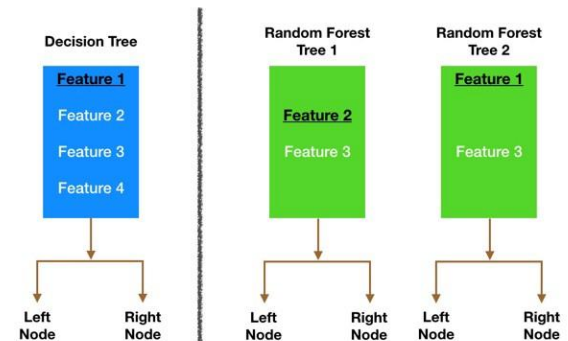
This partitioning may continue along the same attribute to yield smaller and smaller subsets of higher "purity". As we see in the tree above, the dataset is split along the Income attribute at several points.

Random Forrest

- A random forest contains a large number of individual decision trees that form an ensemble. Each individual tree in the random forest generates a class prediction and the class with most votes wins: wisdom of crowds.
- There must be a low correlation between the trees: measure different features. Some trees may be wrong, but probably most will be right.
- Make sure the trees are not too correlated by using:
 - Random bagging of training data.
 - Distribute features randomly over trees



Tally: Six 1s and Three 0s
Prediction: 1



Naïve Bayes Classifiers



Maastricht University

Probabilistic relevance feedback

- Rather than reweighting in a vector space...
- If user has told us some relevant and some irrelevant documents, then we can proceed to build a probabilistic classifier,
 - such as the Naive Bayes model we will look at today:
 - $P(t_k | R) = |\mathbf{D}_{rk}| / |\mathbf{D}_r|$
 - $P(t_k | NR) = |\mathbf{D}_{nrk}| / |\mathbf{D}_{nr}|$
 - t_k is a term; $\mathbf{D}_r$ is the set of known relevant documents; $\mathbf{D}_{rk}$ is the subset that contain t_k ; $\mathbf{D}_{nr}$ is the set of known irrelevant documents; $\mathbf{D}_{nrk}$ is the subset that contain t_k .

Recall a few probability basics

- For events a and b :
- Bayes' Rule

$$p(a, b) = p(a \cap b) = p(a | b) p(b) = p(b | a) p(a)$$

$$p(\bar{a} | b) p(b) = p(b | \bar{a}) p(\bar{a})$$

$$p(a | b) = \frac{p(b | a) p(a)}{p(b)} = \frac{p(b | a) p(a)}{\sum_{x=a, \bar{a}} p(b | x) p(x)}$$

Posterior

Prior

- Odds:

$$O(a) = \frac{p(a)}{p(\bar{a})} = \frac{p(a)}{1 - p(a)}$$

Bayes' Rule for text classification

- For a document d and a class c

$$P(c, d) = P(c \mid d)P(d) = P(d \mid c)P(c)$$

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

Naive Bayes Classifiers

Task: Classify a new instance d based on a tuple of attribute values into one of the classes $c_j \in C$

$$d = \langle x_1, x_2, \dots, x_n \rangle$$

$$c_{MAP} = \operatorname{argmax}_{c_j \in C} P(c_j \mid x_1, x_2, \dots, x_n)$$

$$= \operatorname{argmax}_{c_j \in C} \frac{P(x_1, x_2, \dots, x_n \mid c_j) P(c_j)}{P(x_1, x_2, \dots, x_n)}$$

$$= \operatorname{argmax}_{c_j \in C} P(x_1, x_2, \dots, x_n \mid c_j) P(c_j)$$

MAP is “maximum a posteriori” = most likely class

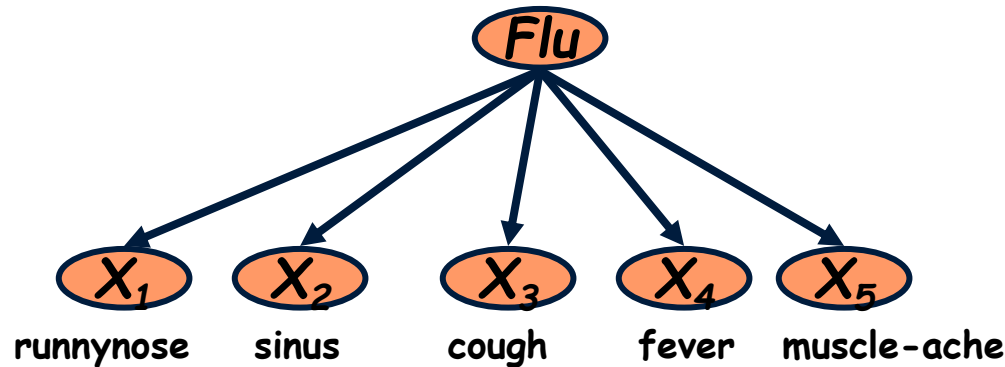
Naive Bayes Classifier: Naïve Bayes Assumption

- $P(c_j)$
 - Can be estimated from the frequency of classes in the training examples.
- $P(x_1, x_2, \dots, x_n | c_j)$
 - $O(|X|^n \cdot |C|)$ parameters
 - Could only be estimated if a very, very large number of training examples was available.

Naive Bayes Conditional Independence Assumption:

- Assume that the probability of observing the conjunction of attributes is equal to the product of the individual probabilities $P(x_i | c_j)$.

The Naive Bayes Classifier

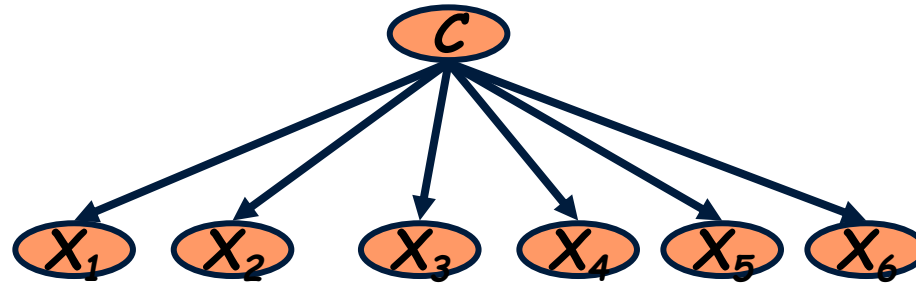


- **Conditional Independence Assumption:** features detect term presence and are **independent** of each other **given the class**:

$$P(X_1, \dots, X_5 | C) = P(X_1 | C) \bullet P(X_2 | C) \bullet \dots \bullet P(X_5 | C)$$

- This model is appropriate for binary variables
 - Multivariate Bernoulli model

Learning the Model

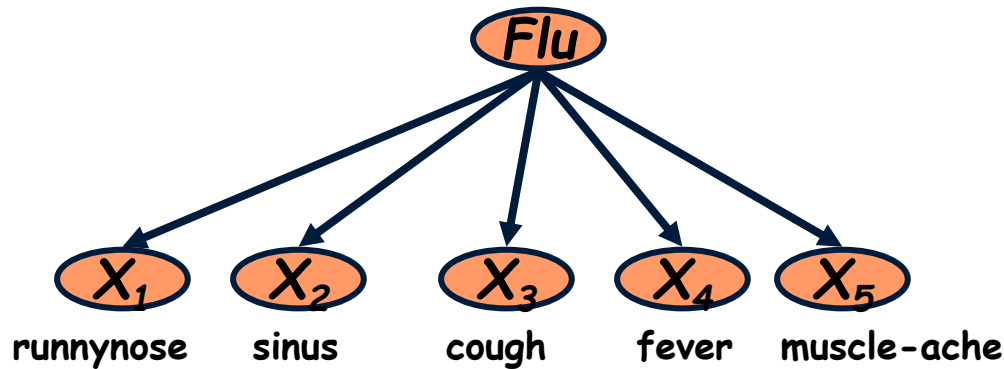


- First attempt: maximum likelihood estimates
 - simply use the frequencies in the data

$$\hat{P}(c_j) = \frac{N(C = c_j)}{N}$$

$$\hat{P}(x_i | c_j) = \frac{N(X_i = x_i, C = c_j)}{N(C = c_j)}$$

Problem with Maximum Likelihood



$$P(X_1, \dots, X_5 \mid C) = P(X_1 \mid C) \bullet P(X_2 \mid C) \bullet \dots \bullet P(X_5 \mid C)$$

- What if we have seen **no training documents** with the word ***muscle-ache*** and classified in the topic ***Flu***?

$$\hat{P}(X_5 = t \mid C = Flu) = \frac{N(X_5 = t, C = Flu)}{N(C = Flu)} = 0$$

- Zero probabilities cannot be conditioned away, no matter the other evidence!**

$$\ell = \arg \max_c \hat{P}(c) \prod_i \hat{P}(x_i \mid c)$$

Smoothing to Avoid Overfitting

$$\hat{P}(x_i | c_j) = \frac{N(X_i = x_i, C = c_j) + 1}{N(C = c_j) + k}$$

of values of X_i

- Somewhat more subtle version

overall fraction in
data where $X_i = x_{i,k}$

$$\hat{P}(x_{i,k} | c_j) = \frac{N(X_i = x_{i,k}, C = c_j) + mp_{i,k}}{N(C = c_j) + m}$$

extent of
“smoothing”¹⁹

Naive Bayes: Learning

- From training corpus, extract *Vocabulary*
- Calculate required $P(c_j)$ and $P(x_k | c_j)$ terms
 - For each c_j in C do
 - $docs_j \leftarrow$ subset of documents for which the target class is c_j
 - $$P(c_j) \leftarrow \frac{|docs_j|}{|\text{total \# documents}|}$$
 - $Text_j \leftarrow$ single document containing all $docs_j$
 - for each word x_k in *Vocabulary*
 - $n_k \leftarrow$ number of occurrences of x_k in $Text_j$
 - $$P(x_k | c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha |Vocabulary|}$$

Naive Bayes: Classifying

- $positions \leftarrow$ all word positions in current document which contain tokens found in *Vocabulary*
- Return c_{NB} , where

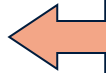
$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in positions} P(x_i | c_j)$$

Naive Bayes: Time Complexity

- **Training Time:** $O(|D|L_{ave} + |C||V|)$

where L_{ave} is the average length of a document in D and V is the size of the Vocabulary (of a document):

- Assumes all counts are pre-computed in $O(|D|L_{ave})$ time during one pass through all of the data.
- Generally just $O(|D|L_{ave})$ since usually $|C||V| < |D|L_{ave}$

 Why?

- **Test Time/Classification:** $O(|C| L_t)$

where L_t is the average length of a test document.

- Very efficient overall, linearly proportional to the time needed to just read in all the data.

Underflow Prevention: using logs

- Multiplying lots of probabilities, which are between 0 and 1 by definition, can result in floating-point underflow.
- Since $\log(xy) = \log(x) + \log(y)$, it is better to perform all computations by summing logs of probabilities rather than multiplying probabilities.
- Class with highest final un-normalized log probability score is still the most probable.

$$c_{NB} = \operatorname{argmax}_{c_j \in C} [\log P(c_j) + \sum_{i \in \text{positions}} \log P(x_i | c_j)]$$

- Note that model is now just max of sum of weights...

Naive Bayes Classifier

$$c_{NB} = \operatorname{argmax}_{c_j \in C} [\log P(c_j) + \sum_{i \in \text{positions}} \log P(x_i | c_j)]$$

- Simple interpretation: Each conditional parameter $\log P(x_i | c_j)$ is a weight that indicates how good an indicator x_i is for c_j .
- The prior $\log P(c_j)$ is a weight that indicates the relative frequency of c_j .
- The sum is then a measure of how much evidence there is for the document being in the class.
- We select the class with the most evidence for it

Recap: Naïve Bayes classifiers

- Classify based on prior weight of class and conditional parameter for what each word says:

$$c_{NB} = \operatorname{argmax}_{c_j \in C} \left[\log P(c_j) + \sum_{i \in \text{positions}} \log P(x_i | c_j) \right]$$

- Training is done by counting and dividing:

$$P(c_j) \leftarrow \frac{N_{c_j}}{N}$$

- Don't forget to smooth

$$P(x_k | c_j) \leftarrow \frac{T_{c_j x_k} + \alpha}{\sum_{x_i \in V} [T_{c_j x_i} + \alpha]}$$

Feature Selection

Overcoming dependencies
Select the most discriminating features

χ^2 statistic (CHI)

- χ^2 is interested in $(f_o - f_e)^2 / f_e$ summed over all table entries: is the **observed** (O) number what you'd **expect** (E) given the marginals?

$$\chi^2(j, a) = \sum (O - E)^2 / E = (2 - .25)^2 / .25 + (3 - 4.75)^2 / 4.75 \\ + (500 - 502)^2 / 502 + (9500 - 9498)^2 / 9498 = 12.9 \quad (p < .001)$$

- The null hypothesis (is freq distribution observed in line with theoretical freq distribution) is rejected with confidence .999, since $12.9 > 10.83$ (the value for .999 confidence).

	<i>Term = jaguar</i>	<i>Term ≠ jaguar</i>	
<i>Class = auto</i>	2 (0.25)	500 (502)	expected: f_e
<i>Class ≠ auto</i>	3 (4.75)	9500 (9498)	observed: f_o

Feature selection via Mutual Information

- In training set, choose k words which best discriminate (give most info on) the categories.
- The Mutual Information between a word, class is:

$$I(w, c) = \sum_{e_w \in \{0,1\}} \sum_{e_c \in \{0,1\}} p(e_w, e_c) \log \frac{p(e_w, e_c)}{p(e_w)p(e_c)}$$

- For each word w and each category c

Feature selection via MI (contd.)

- For each category we build a list of k most discriminating terms.
- For example (on 20 Newsgroups):
 - ***sci.electronics***: circuit, voltage, amp, ground, copy, battery, electronics, cooling, ...
 - ***rec.autos***: car, cars, engine, ford, dealer, mustang, oil, collision, autos, tires, toyota, ...
- Does not account for correlations between terms
- Why?

Feature selection for NB & LR

- In general feature selection is *necessary* NB and for Logistic Regression (up next after the break!).
- Otherwise, you suffer from noise, multi-counting

Disadvantage Feature Selection

- The classifier will be highly optimized for a particular data set.
- The classifier will not perform as well on documents not included in the feature selection process.
- No or limited transfer learning possible.

Evaluation



Maastricht University

Naive Bayes vs. other methods

(a)	NB	Rocchio	kNN	SVM	
micro-avg-L (90 classes)	80	85	86	89	
macro-avg (90 classes)	47	59	60	60	

(b)	NB	Rocchio	kNN	trees	SVM
earn	96	93	97	98	98
acq	88	65	92	90	94
money-fx	57	47	78	66	75
grain	79	68	82	85	95
crude	80	70	86	85	89
trade	64	65	77	73	76
interest	65	63	74	67	78
ship	85	49	79	74	86
wheat	70	69	77	93	92
corn	65	48	78	92	90
micro-avg (top 10)	82	65	82	88	92
micro-avg-D (118 classes)	75	62	n/a	n/a	87

Evaluation measure: F_1

Naive Bayes does pretty well, but some methods beat it consistently (e.g., SVM).

Violation of NB Assumptions

- The independence assumptions do not really hold of documents written in natural language.
 - Conditional independence
 - Positional independence

Logistic Regression



Maastricht University

Logistic Regression

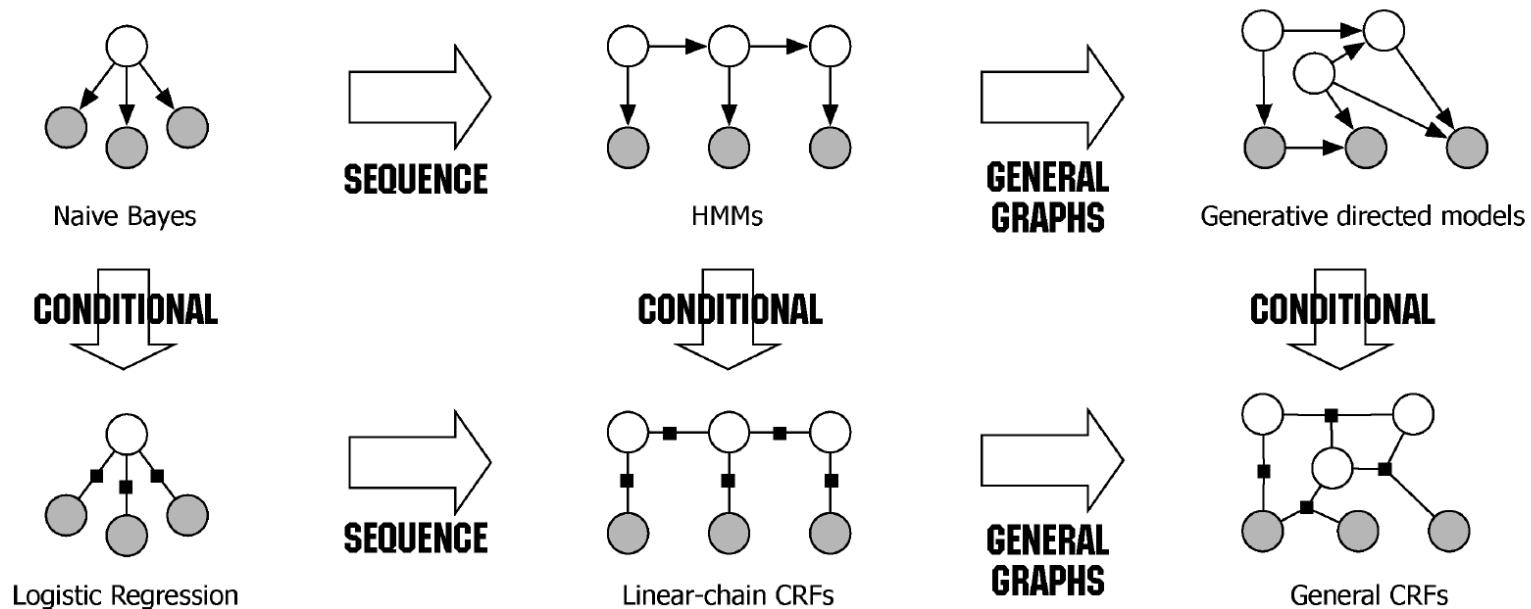


Fig. 2.4 Diagram of the relationship between naive Bayes, logistic regression, HMMs, linear-chain CRFs, generative models, and general CRFs.

Logistic Regression vs Naïve Bayes

Naive Bayes is a generative classifier -

- **Logistic Regression** is a **discriminative classifier**

Suppose two classes, document is *invoice* or document is a *memo*

- Generative: given a sample text, ask the system what is most likely: *invoice* or *memo*? The most likely one is chosen.
- **Discriminative**: learns by distinguishing classes by features. E.g.
 - an invoice has more numbers than a memo,
 - an invoice has columns and rows,
 - memo is more continuous text: longer sentences,
 - ...

Generative versus discriminative model

- Naïve Bayes assigns a class c to a document d by using:

$$\hat{c} = \operatorname{argmax}_{c \in C} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}$$

- A discriminative model attempts to directly compute $P(c|d)$. Does this by assigning weights to discriminative features. Does not generate an example of a class. Learns by supervised machine learning, not by estimating probability distributions.

Logistic Regression

1. A **feature representation** of the input. For each input observation $x^{(i)}$, this will be a vector of features $[x_1, x_2, \dots, x_n]$. We will generally refer to feature i for input $x^{(j)}$ as $x_i^{(j)}$, sometimes simplified as x_i , but we will also see the notation f_i , $f_i(x)$, or, for multiclass classification, $f_i(c, x)$.
2. A classification function that computes $\hat{y}$, the estimated class, via $p(y|x)$. In the next section we will introduce the **sigmoid** and **softmax** tools for classification.
3. An objective function for learning, usually involving minimizing error on training examples. We will introduce the **cross-entropy loss function**
4. An algorithm for optimizing the objective function. We introduce the **stochastic gradient descent** algorithm.

Logistic regression has two phases:

training: we train the system (specifically the weights w and b) using stochastic gradient descent and the cross-entropy loss.

test: Given a test example x we compute $p(y|x)$ and return the higher probability label $y = 1$ or $y = 0$.

1. Features are for instance the presence of certain (pre-selected) words ($x_1, x_2, x_3, \dots, x_n$) or whatever feature function (same as with CRF and MEMM).
2. Classification function is done by using the function $z = wx + b$ where w is weights set at training and x is feature vector and b is a bias. The output y can be estimated by using a sigmoid function as below:

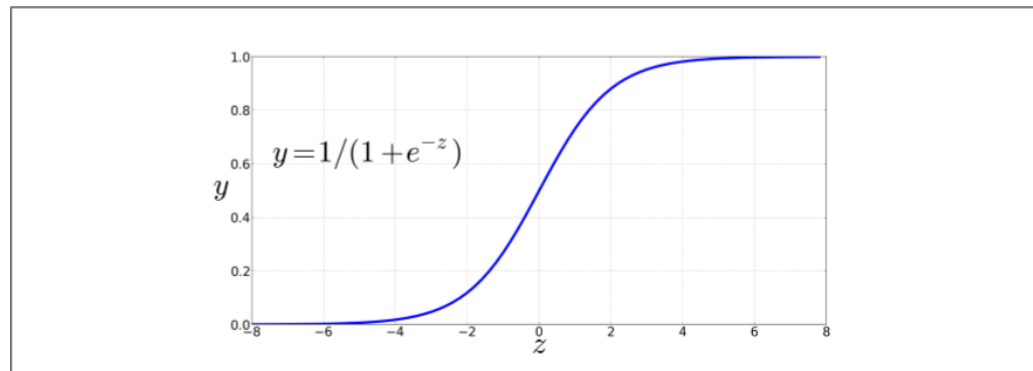


Figure 5.1 The sigmoid function $y = \frac{1}{1+e^{-z}}$ takes a real value and maps it to the range $[0, 1]$. It is nearly linear around 0 but outlier values get squashed toward 0 or 1.

Logistic Regression: Classifying

- As we deal with probabilities: the sum of $p(y=1)$ and $p(y=0)$ must be 1. This allows us to calculate the probabilities as follows. We could for instance take $p(y=1) > 0.5^*$ as a threshold for decision:

$$\begin{aligned} P(y = 1) &= \sigma(w \cdot x + b) \\ &= \frac{1}{1 + e^{-(w \cdot x + b)}} \end{aligned}$$

* This is named the “decision boundary”

$$\begin{aligned} P(y = 0) &= 1 - \sigma(w \cdot x + b) \\ &= 1 - \frac{1}{1 + e^{-(w \cdot x + b)}} \\ &= \frac{e^{-(w \cdot x + b)}}{1 + e^{-(w \cdot x + b)}} \end{aligned}$$

Logistic regression: training

- Estimating weights w and bias b .
- How “close” is the classification of the current label to the supervised gold true. Rather than using a measure similarity, we usually talk about the opposite of this: a loss or cost function, the distance between what we should observe and what we observe.

$$L(\hat{y}, y) = \text{How much } \hat{y} \text{ differs from the true } y$$

See: <https://web.stanford.edu/~jurafsky/slp3/5.pdf> for details of derivation.

$$L_{CE}(w, b) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

Next, we need an iterative algorithm that slowly minimizes the loss function by updating the weights: *gradient descent*.



Gradient Descent

- Goal of Gradient Descent is to find the optimal weights: minimize the loss function for the model. So minimize following function:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{CE}(y^{(i)}, x^{(i)}; \theta)$$

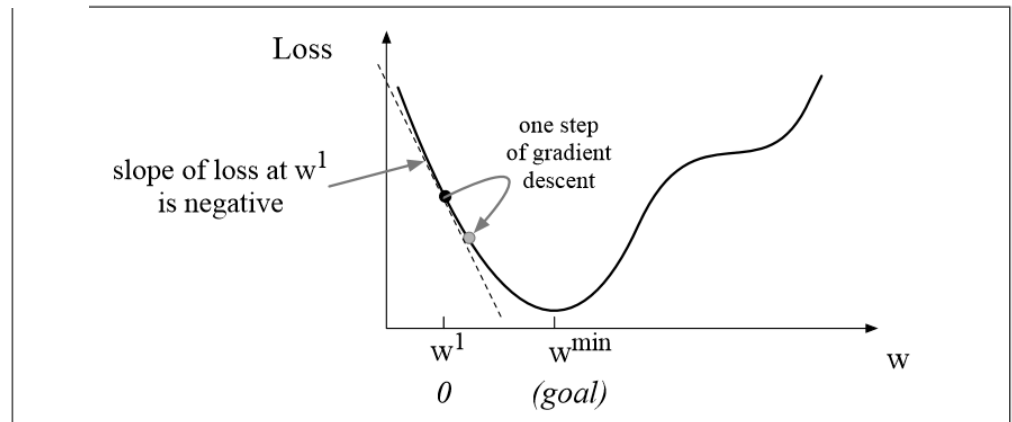
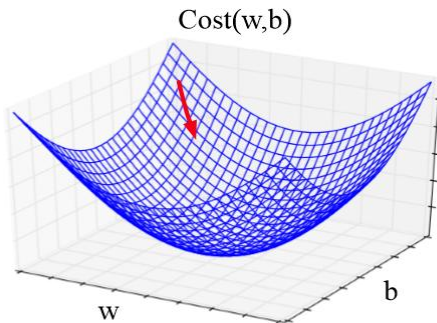


Figure 5.3 The first step in iteratively finding the minimum of this loss function, by moving w in the reverse direction from the slope of the function. Since the slope is negative, we need to move w in a positive direction, to the right. Here superscripts are used for learning steps, so w^1 means the initial value of w (which is 0), w^2 at the second step, and so on.

Gradient Descent learning rate

We have a learning rate η in:

$$w^{t+1} = w^t - \eta \frac{d}{dw} f(x; w)$$

Updating function:

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

All looks very complicated, but it turns out that the derivative of the cross-entropy function $L_{CE}(w, b)$ for one observation is a simple formula hereunder. We call this the “gradient descent for logistic regression”

$$\nabla_{\theta} L(f(x; \theta), y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x; \theta), y) \\ \frac{\partial}{\partial w_2} L(f(x; \theta), y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x; \theta), y) \end{bmatrix}$$

$$\frac{\partial L_{CE}(w, b)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

See: <https://web.stanford.edu/~jurafsky/slp3/5.pdf> for details of derivation.

Stochastic Gradient Descent Algorithm

```
function STOCHASTIC GRADIENT DESCENT( $L()$ ,  $f()$ ,  $x$ ,  $y$ ) returns  $\theta$ 
    # where:  $L$  is the loss function
    #  $f$  is a function parameterized by  $\theta$ 
    #  $x$  is the set of training inputs  $x^{(1)}, x^{(2)}, \dots, x^{(n)}$ 
    #  $y$  is the set of training outputs (labels)  $y^{(1)}, y^{(2)}, \dots, y^{(n)}$ 

     $\theta \leftarrow 0$ 
    repeat til done # see caption
        For each training tuple  $(x^{(i)}, y^{(i)})$  (in random order)
            1. Optional (for reporting): # How are we doing on this tuple?
                Compute  $\hat{y}^{(i)} = f(x^{(i)}; \theta)$  # What is our estimated output  $\hat{y}$ ?
                Compute the loss  $L(\hat{y}^{(i)}, y^{(i)})$  # How far off is  $\hat{y}^{(i)}$  from the true output  $y^{(i)}$ ?
            2.  $g \leftarrow \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$  # How should we move  $\theta$  to maximize loss?
            3.  $\theta \leftarrow \theta - \eta g$  # Go the other way instead
    return  $\theta$ 
```

Figure 5.5 The stochastic gradient descent algorithm. Step 1 (computing the loss) is used to report how well we are doing on the current tuple. The algorithm can terminate when it converges (or when the gradient $< \epsilon$), or when progress halts (for example when the loss starts going up on a held-out set).

Logistic Regression

- Simple algorithms that works well as a baseline. Computationally efficient. Highly interpretable (transparent).
- All feature functions are binary: so you cannot use TF-IDF for instance if the presence of each word is an automatic feature function. Creating feature functions require quite some effort if anything else chosen than just presence of key words.
- Requires extensive feature selection by removing features that are not relevant to the output. Does not do implicit feature selection. You must identify all important independent variables in advance.
- Cannot solve non-linear problems.
- Very sensitive to class imbalance. Vulnerable to overfitting.

K-NN

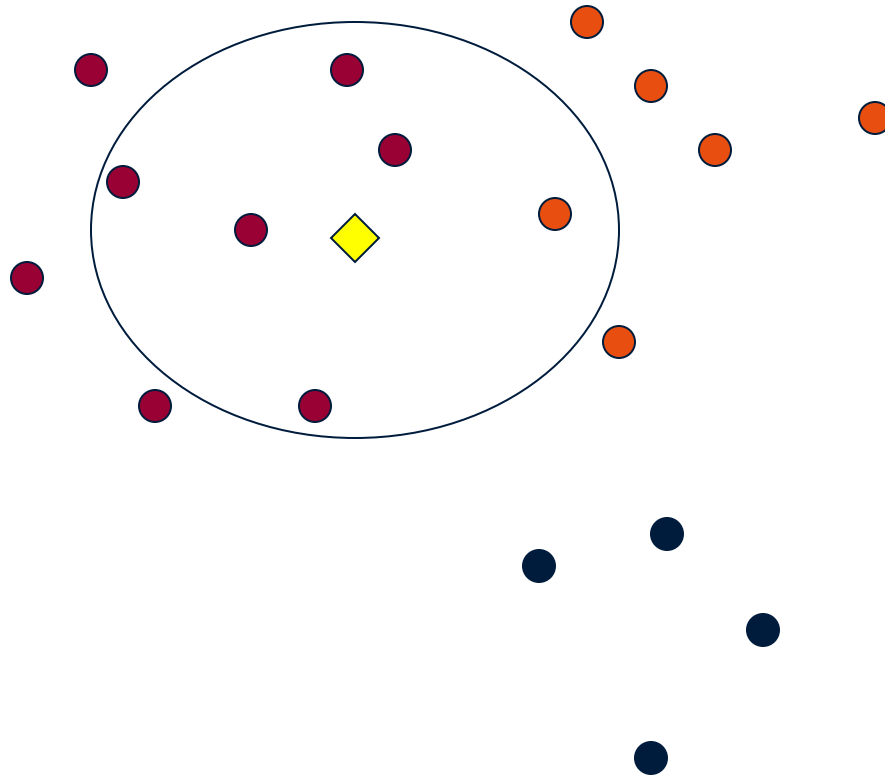


Maastricht University

k Nearest Neighbor Classification

- kNN = k Nearest Neighbor
- To classify a document d into class c :
- Define k -neighborhood N as k nearest neighbors of d
- Count number of documents i in N that belong to c
- Estimate $P(c|d)$ as i/k
- Choose as class $\text{argmax}_c P(c|d)$ [= majority class]

Example: $k=6$ (6NN)



$P(\text{science} | \text{diamond})?$

- Government
- Science
- Arts

Nearest-Neighbor Learning Algorithm

- Learning is just storing the representations of the training examples in D .
- Testing instance x (*under 1NN*):
 - Compute similarity between x and all examples in D .
 - Assign x the category of the most similar example in D .
- Does not explicitly compute a generalization or category prototypes.
- Also called:
 - Case-based learning
 - Memory-based learning
 - Lazy learning
- Rationale of kNN: contiguity hypothesis

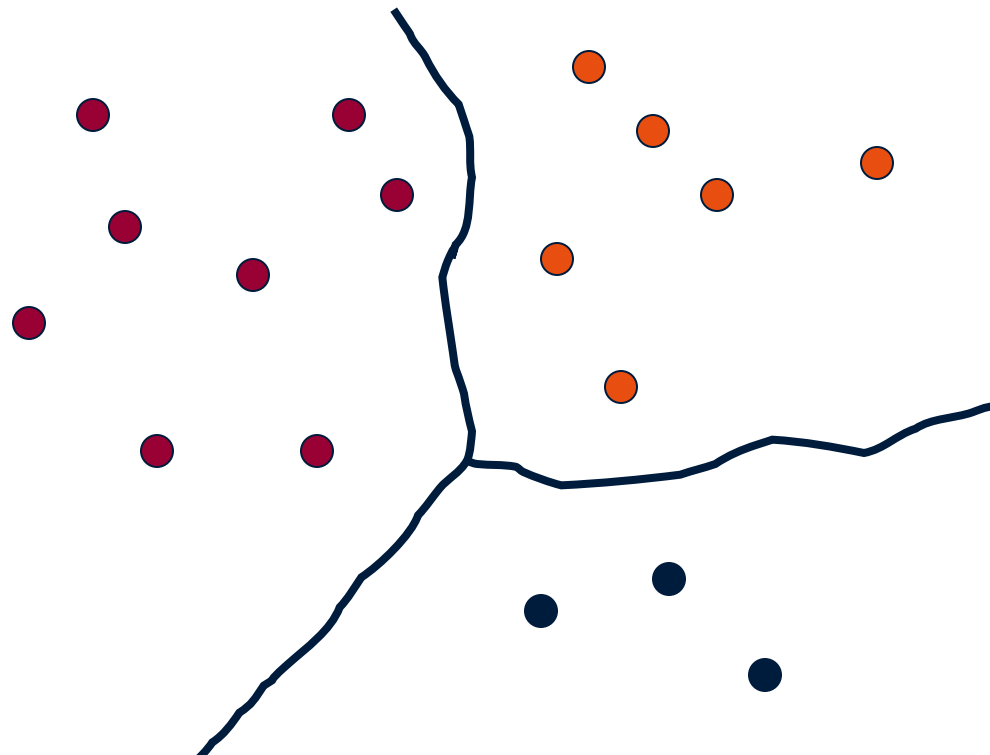
kNN Is Close to Optimal

- Cover and Hart (1967)
- Asymptotically, the error rate of 1-nearest-neighbor classification is less than twice the Bayes rate [error rate of classifier knowing model that generated data]
- In particular, asymptotic error rate is 0 if Bayes rate is 0.
- Assume: query point coincides with a training point.
- Both query point and training point contribute error $\rightarrow$ 2 times Bayes rate

k Nearest Neighbor

- Using only the closest example (1-NN) to determine the class is subject to errors due to:
 - A single atypical example.
 - Noise (i.e., an error) in the category label of a single training example.
- More robust alternative is to find the k most-similar examples and return the majority category of these k examples.
- Value of k is typically odd to avoid ties; 3 and 5 are most common.

kNN decision boundaries



Boundaries are in principle arbitrary surfaces – but usually polyhedra

- Government
- Science
- Arts

kNN gives locally defined decision boundaries between classes – far away points do not influence each classification decision (unlike in Naïve Bayes, Rocchio, etc.)

Similarity Metrics

- Nearest neighbor method depends on a similarity (or distance) metric.
- Simplest for continuous m -dimensional instance space is *Euclidean distance*.
- Simplest for m -dimensional binary instance space is *Hamming distance* (number of feature values that differ).
- For text, cosine similarity of TF-IDF weighted vectors is typically most effective.
- Or: Levenshtein, Q-distance, ...

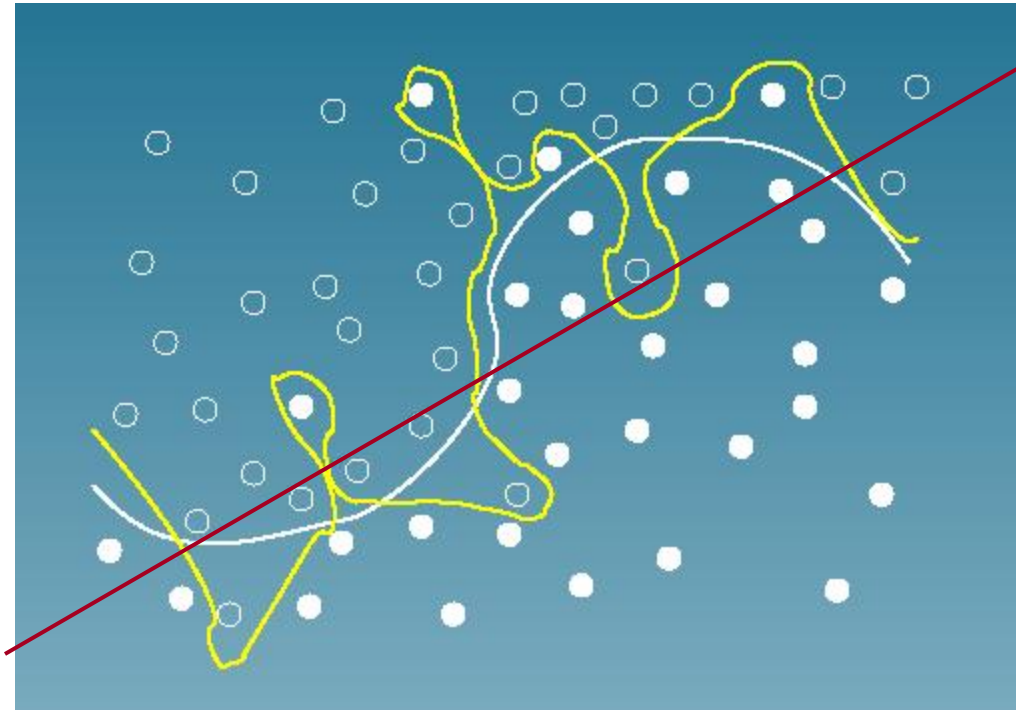
kNN: Discussion

- No feature selection necessary
- Scales well with large number of classes
 - Don't need to train n classifiers for n classes
- Classes can influence each other
 - Small changes to one class can have ripple effect
- Scores can be hard to convert to probabilities
- No training necessary
 - Actually: perhaps not true. (Data editing, etc.)
- May be expensive at test time
- In most cases it's more accurate than NB or Rocchio

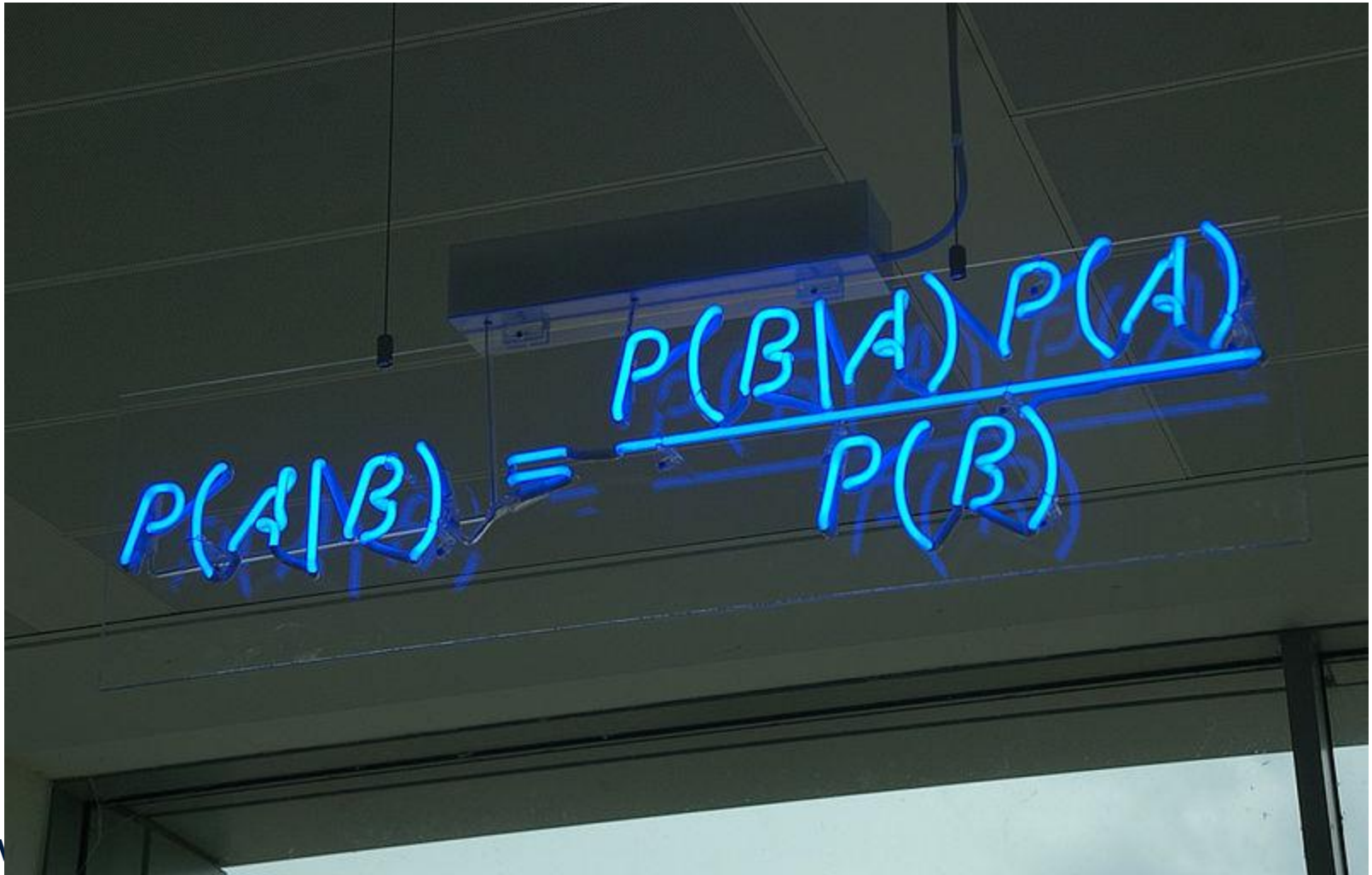
kNN vs. Naive Bayes

- Bias/Variance tradeoff
 - Variance $\approx$ Capacity
- kNN has **high variance** and **low bias**.
 - Infinite memory
- NB has **low variance** and **high bias**.
 - Decision surface has to be linear (hyperplane – see later)
- Consider asking a botanist: **Is an object a tree?**
 - Too much capacity/variance, low bias
 - Botanist who memorizes
 - Will always say “no” to new object (e.g., different # of leaves)
 - Not enough capacity/variance, high bias
 - Lazy botanist
 - Says “yes” if the object is green
 - You want the middle ground

Bias vs. variance: Choosing the correct model capacity



Break



A blue neon sign is mounted on a dark, textured wall. The sign displays the probability formula for conditional probability: $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$. The text is written in a stylized, glowing blue font. The sign is illuminated, casting a soft glow on the surrounding wall.

Linear and non-linear problems



Maastricht University

Linear Classifiers

- Many common text classifiers are linear classifiers
 - Naïve Bayes
 - Perceptron
 - Rocchio
 - Logistic regression
 - Support vector machines (with linear kernel)
 - Linear regression with threshold
- Despite this similarity, noticeable performance differences
 - For separable problems, there is an infinite number of separating hyperplanes. Which one do you choose?
 - What to do for non-separable problems?
 - Different training methods pick different hyperplanes
- Classifiers more powerful than linear often don't perform better on text problems. Why?

Naive Bayes is a linear classifier

- Two-class Naive Bayes. We compute:

$$\log \frac{P(C | d)}{P(\bar{C} | d)} = \log \frac{P(C)}{P(\bar{C})} + \sum_{w \in d} \log \frac{P(w | C)}{P(w | \bar{C})}$$

- Decide class C if the odds is greater than 1, i.e., if the log odds is greater than 0.
- So, decision boundary is hyperplane:

$$\alpha + \sum_{w \in V} \beta_w \times n_w = 0 \quad \text{where } \alpha = \log \frac{P(C)}{P(\bar{C})};$$

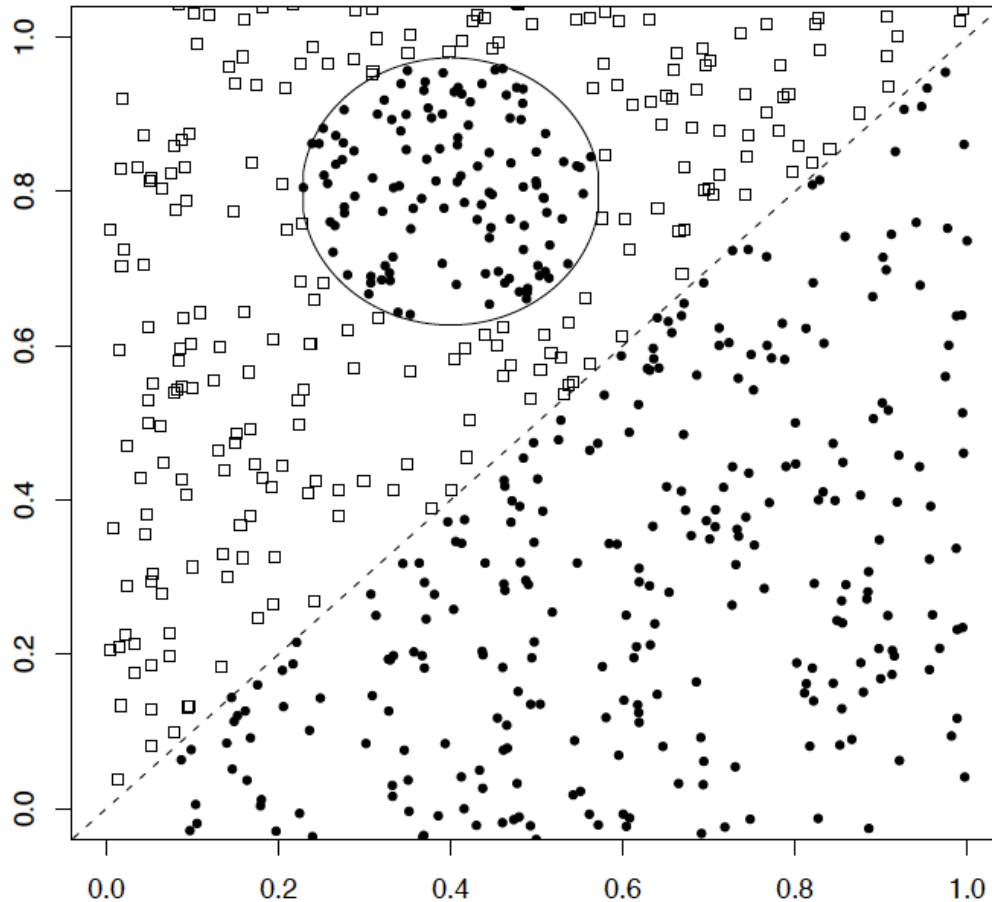
$$\beta_w = \log \frac{P(w | C)}{P(w | \bar{C})}; \quad n_w = \# \text{ of occurrences of } w \text{ in } d$$

Logistic Regression is a linear classifier

- Logistic regression is considered a generalized linear model because the outcome always depends on the sum of the inputs and parameters; the output cannot depend on the product (or quotient, etc.) of its parameters, so it is linear.

$$z = w_0x_0 + w_1x_1 + \dots + w_mx_m = \sum_{j=1}^m w_jx_j = \mathbf{w}^T \mathbf{x}.$$

A nonlinear problem



- A linear classifier like Naïve Bayes does badly on this task
- kNN will do very well (assuming enough training data)
- So, always ask yourself if your classifier model fits your real-world problem!

Support Vector Machines (SVM)

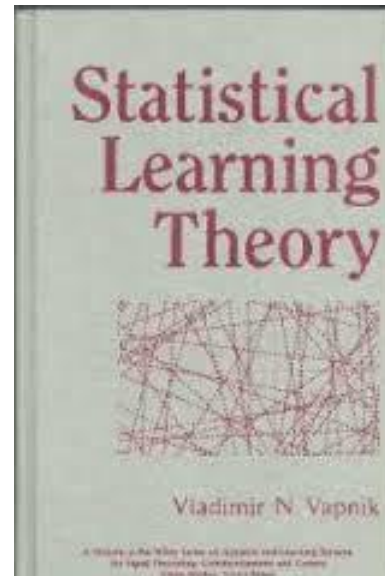
Linear
Polynomial
Gaussian
Other Kernels



Maastricht University

History: Support Vector Machines

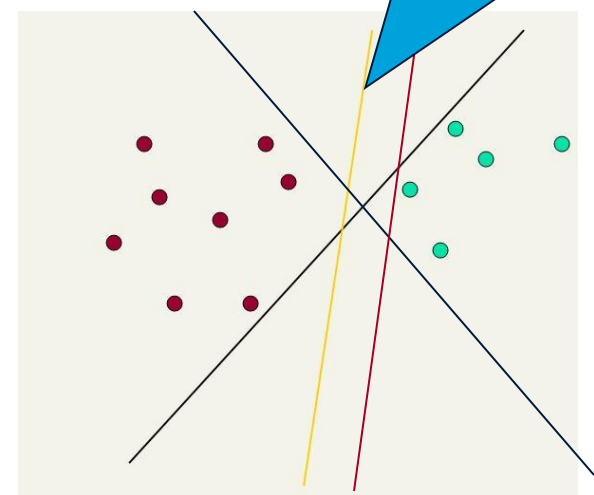
The original SVM algorithm was invented by Vladimir N. Vapnik and the current standard incarnation (soft margin) was proposed by Corinna Cortes and Vapnik in 1993 and published 1995: Cortes, C.; Vapnik, V. (1995). "Support-vector networks". Machine Learning 20 (3): 273.



Linear classifiers: Which Hyperplane?

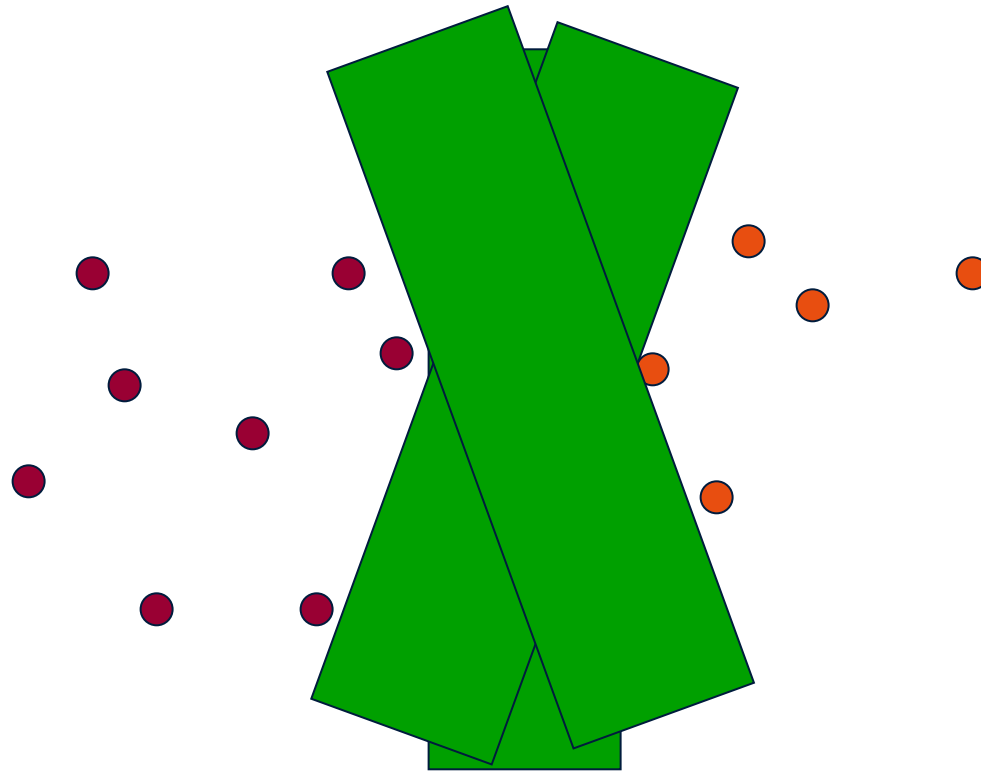
- Lots of possible solutions for a , b , c .
- Some methods find a separating hyperplane, but not the optimal one [according to some criterion of expected goodness]
 - E.g., perceptron
- Support Vector Machine (SVM) finds an optimal solution.
 - Maximizes the distance between the hyperplane and the “difficult points” close to decision boundary
 - One intuition: if there are no points near the decision surface, then there are no very uncertain classification decisions

This line represents the decision boundary:
 $ax + by - c = 0$



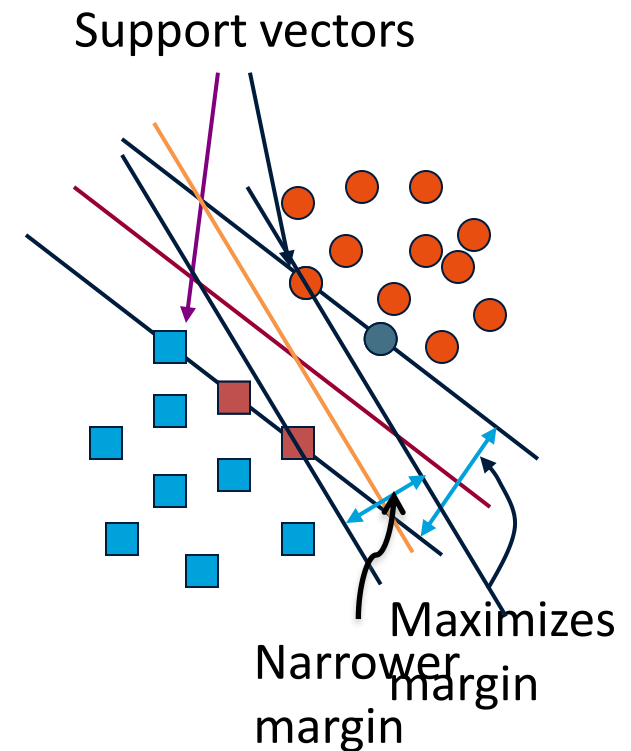
Another intuition

- If you have to place a fat separator between classes, you have less choices, and so the capacity of the model has been decreased



Support Vector Machine (SVM)

- SVMs maximize the *margin* around the separating hyperplane.
 - A.k.a. large margin classifiers
- The decision function is fully specified by a subset of training samples, *the support vectors*.
- Solving SVMs is a *quadratic programming* problem
- Currently widely seen as as the best text classification method.



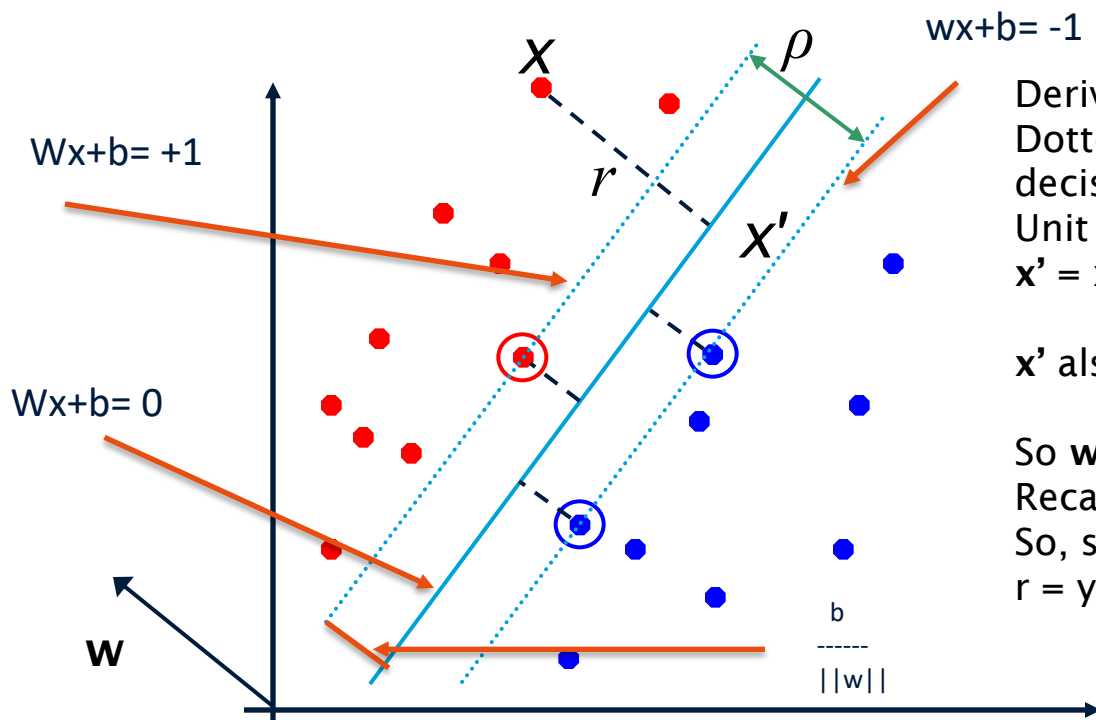
Maximum Margin: Formalization

- $\mathbf{w}$: decision hyperplane normal vector
- $\mathbf{x}_i$: data point i
- y_i : class of data point i (+1 or -1) Note: Not 1/0
- $\mathbf{b}$: margin from lower or upper line to middle line. $2xb$ = full margin between data points
- Classifier is: $f(\mathbf{x}_i) = \text{sign}(\mathbf{w}^T \mathbf{x}_i + b)$
- Functional margin of $\mathbf{x}_i$ is: $y_i (\mathbf{w}^T \mathbf{x}_i + b)$
 - But note that we can increase this margin simply by scaling $\mathbf{w}$, $\mathbf{b}$
- Functional margin of dataset is twice the minimum functional margin for any point
 - The factor of 2 comes from measuring the whole width of the margin (related to b)

Mechanics of SVM: Geometric Margin

- We can derive distance r from example X to the separator:
- Examples closest to the hyperplane are **support vectors**.
- Margin** ρ of the separator is the width of separation between support vectors of classes.

$$r = y \frac{\mathbf{w}^T \mathbf{x} + b}{\|\mathbf{w}\|}$$



Derivation of finding r :
 Dotted line $\mathbf{x}' - \mathbf{x}$ is orthogonal to decision boundary so parallel to $\mathbf{w}$.
 Unit vector is $\mathbf{w}/\|\mathbf{w}\|$, $r = \text{line } \mathbf{x}\mathbf{x}' = r\mathbf{w}/\|\mathbf{w}\|$
 $\mathbf{x}' = \mathbf{x} - \mathbf{x}\mathbf{x}' = \mathbf{x} - yr\mathbf{w}/\|\mathbf{w}\|$ with $y = +/ -1$

$\mathbf{x}'$ also satisfies $\mathbf{w}^T \mathbf{x}' + b = 0$.

So $\mathbf{w}^T (\mathbf{x} - yr\mathbf{w}/\|\mathbf{w}\|) + b = 0$
 Recall that $\|\mathbf{w}\| = \sqrt{\mathbf{w}^T \mathbf{w}}$.
 So, solving for r gives:
 $r = y(\mathbf{w}^T \mathbf{x} + b)/\|\mathbf{w}\|$

Linear SVM Mathematically

The linearly separable case

- Assuming all the data has a functional margin of at least 1, the following two constraints follow for a training set $\{(\mathbf{x}_i, y_i)\}$

$$\mathbf{w}^T \mathbf{x}_i + b \geq 1 \quad \text{if } y_i = 1 \text{ (positive examples)}$$

$$\mathbf{w}^T \mathbf{x}_i + b \leq -1 \quad \text{if } y_i = -1 \text{ (negative examples)}$$

- For support vectors, the inequality becomes an equality.

$$\mathbf{w}^T \mathbf{x}_i + b = 1 \quad \text{if } y_i = 1 \text{ (positive examples)}$$

$$\mathbf{w}^T \mathbf{x}_i + b = -1 \quad \text{if } y_i = -1 \text{ (negative examples)}$$

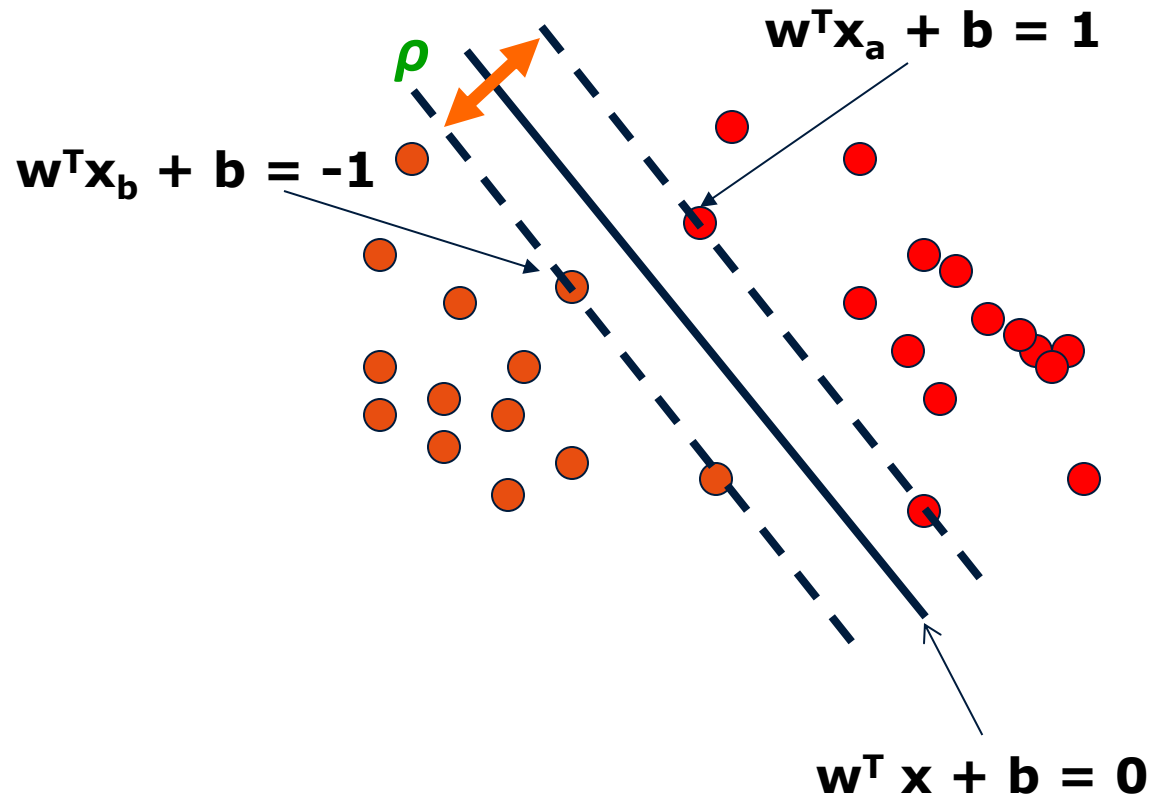
- Then, since each example's distance from the hyperplane is

$$r = y \frac{\mathbf{w}^T \mathbf{x} + b}{\|\mathbf{w}\|}$$

- The margin is: $\rho = \frac{2}{\|\mathbf{w}\|}$

Linear Support Vector Machine (SVM)

- **Hyperplane**
 $w^T x + b = 0$
- **Extra scale constraint:**
 $\min_{i=1,\dots,n} |w^T x_i + b| = 1$
- This implies:
 $w^T(x_a - x_b) = 2$
 $\rho = 2/||w||$



Linear SVMs Mathematically (cont.)

- Then we can formulate the *quadratic optimization problem*:

Find $\mathbf{w}$ and b such that

$$\rho = \frac{2}{\|\mathbf{w}\|} \quad \text{is maximized; and for all } \{(\mathbf{x}_i, y_i)\}$$

$$\mathbf{w}^T \mathbf{x}_i + b \geq 1 \text{ if } y_i = 1; \quad \mathbf{w}^T \mathbf{x}_i + b \leq -1 \text{ if } y_i = -1$$

- A better formulation ($\min \|\mathbf{w}\| = \max 1/\|\mathbf{w}\|$):

Find $\mathbf{w}$ and b such that

$$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} \quad \text{is minimized;}$$

$$\text{and for all } \{(\mathbf{x}_i, y_i)\}: \quad y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

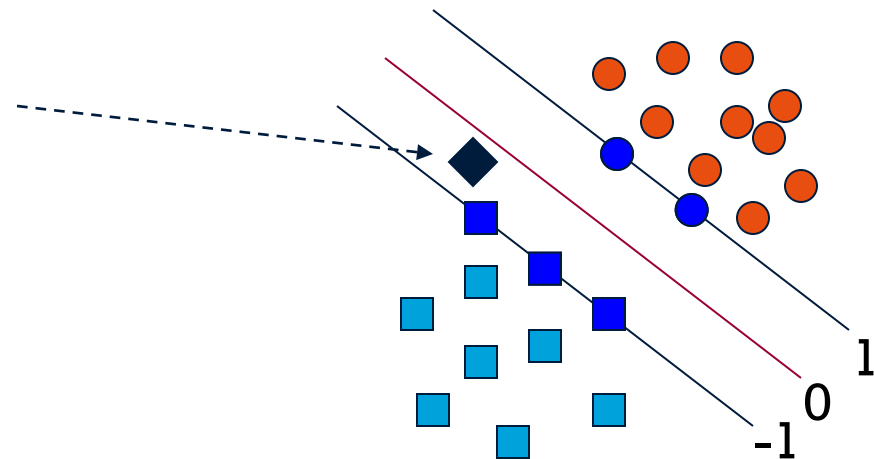
Classification with SVMs

- Given a new point $\mathbf{x}$, we can score its projection onto the hyperplane normal:
 - I.e., compute score: $\mathbf{w}^T \mathbf{x} + b = \sum \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$
 - Can set confidence threshold t .

Score $> t$: yes

Score $< -t$: no

Else: don't know



Linear SVMs: Summary

- The classifier is a *separating hyperplane*.
- The most “important” training points are the support vectors; they define the hyperplane.
- Quadratic optimization algorithms can identify which training points $\mathbf{x}_i$ are support vectors with non-zero Lagrangian multipliers α_i .
- Both in the dual formulation of the problem and in the solution, training points appear only inside inner products:

Find $\alpha_1 \dots \alpha_N$ such that

$Q(\boldsymbol{\alpha}) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$ is maximized and

(1) $\sum \alpha_i y_i = 0$

(2) $0 \leq \alpha_i \leq C$ for all α_i

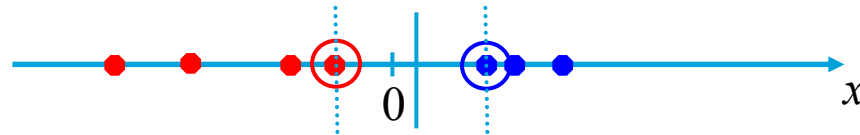
Training

$$f(\mathbf{x}) = \sum \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$$

Classification

Non-linear SVMs

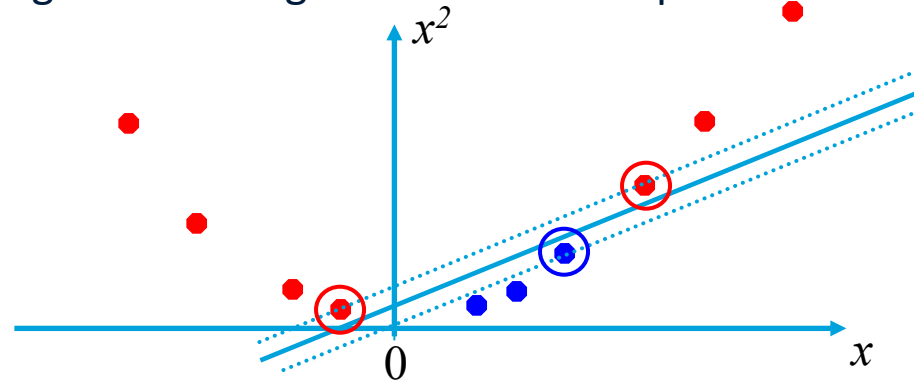
- Datasets that are linearly separable (with some noise) work out great:



- But what are we going to do if the dataset is just too hard?

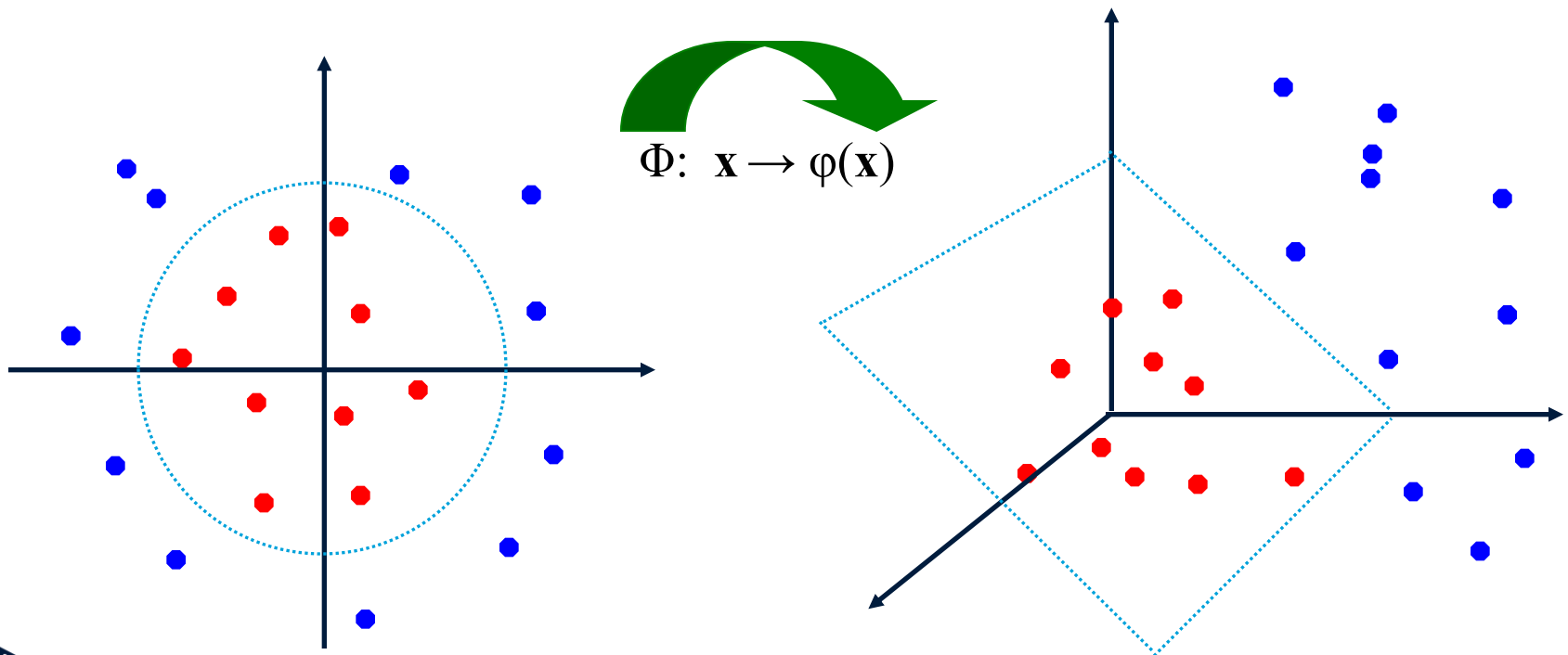


- How about ... mapping data to a higher-dimensional space:



Non-linear SVMs: Feature spaces

- General idea: the original feature space can always be mapped to some higher-dimensional feature space where the training set is separable:



Remember TF-IDF for Document Representation

- Very sparse Data Representation
- So, we can use linear SVM



$$w_{i,j} = tf_{i,j} \times \log \left(\frac{N}{df_i} \right)$$

tf_{ij} = number of occurrences of i in j
 df_i = number of documents containing i
 N = total number of documents

TF-IDF and LSVM are superior for (long) text classification

(a)	NB	Rocchio	kNN	SVM
micro-avg-L (90 classes)	80	85	86	89
macro-avg (90 classes)	47	59	60	60

(b)	NB	Rocchio	kNN	trees	SVM
earn	96	93	97	98	98
acq	88	65	92	90	94
money-fx	57	47	78	66	75
grain	79	68	82	85	95
crude	80	70	86	85	89
trade	64	65	77	73	76
interest	65	63	74	67	78
ship	85	49	79	74	86
wheat	70	69	77	93	92
corn	65	48	78	92	90
micro-avg (top 10)	82	65	82	88	92
micro-avg-D (118 classes)	75	62	n/a	n/a	87

Evaluation measure: F_1

Class In-balance

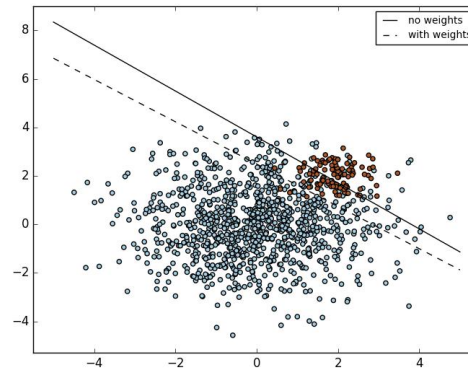


Maastricht University

Dealing with Class Imbalance

- Class Imbalance: only 0.1% of all documents fits in a class.
- SVMs produce suboptimal results with imbalanced datasets.
- The SVM classifier will be biased towards the majority class and have low performance on the minority class.

The hyperplane skews towards the minority class, reducing the performance on the minority class.



- Data preprocessing and algorithmic techniques can solve this problem.

External and internal imbalance learning methods for SVMs.

- *External methods* use preprocessing techniques on the data used by the classification algorithm.
 - Sampling Methods
 - Ensemble Methods
- *Internal methods* modify the classification algorithm itself to handle the class imbalance.

External Sampling Methods for SVM Imbalance

Sampling methods modify the composition of the training set by over- or under sampling.

- **Under sampling** takes a random subset of the majority class to balance the training set with a equal number of positive and negative samples. The training set is reduced, which results in faster training and makes it a more efficient method. However, because not all samples of the majority class are used for training, this information is neglected.
- **Over sampling** creates a more balanced training set by repeating minority samples. This results in a larger training set and increased training time.

External Ensemble Methods for SVM Imbalance

- **Ensemble learning:** split the majority class into multiple non-overlapping subsets so that each of the subsets becomes balanced compared to the minority class. Each classifier is then trained on the subset of the majority class and all the samples of the minority class.
- Other ensemble methods used to handle class imbalance are based on **Boosting**. Weak learner classifiers are iteratively combined where higher weights are assigned to instances that were misclassified in a previous iteration.

Internal methods for SVM Imbalance

- Internal methods introduce modifications to the SVM algorithm itself to make it less sensitive to class imbalance.
- By assigning a higher misclassification cost to the minority class, the separating hyperplane becomes less skewed towards the minority class.

See also: Akbani, R., Kwek, S., & Japkowicz, N. (2004). Applying support vector machines to imbalanced datasets. In Machine learning: Ecml 2004, pp. 39-50, Springer.



Deep Learning & Text-Classification

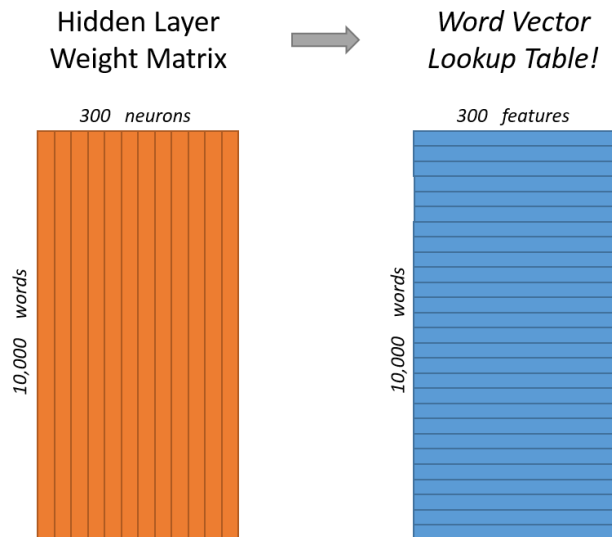


Maastricht University

How about Word Embeddings?

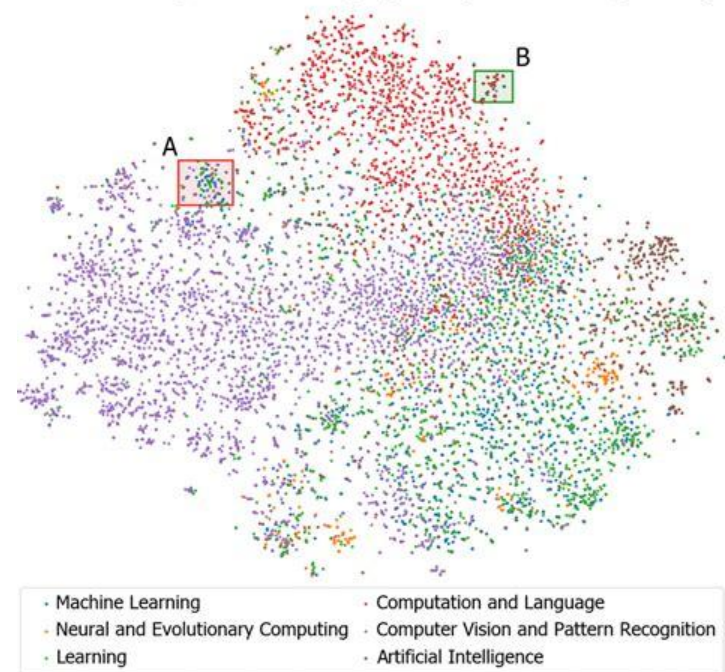
- Pre-trained model
- Understand context better
- Transfer learning: understand already general aspects of language, subsequent only need to fine-tune for a specific NLP task.
- No need for millions or billions of annotated training data (when using deep learning).

Word Embeddings & SVM



No longer a sparse data representation. Millions of words in 300 bits

Embeddings for arXiv papers (6 ML categories)



Can no longer find **one dimension** that separates positive from negative data points

Word2Vec

Dense Matrix

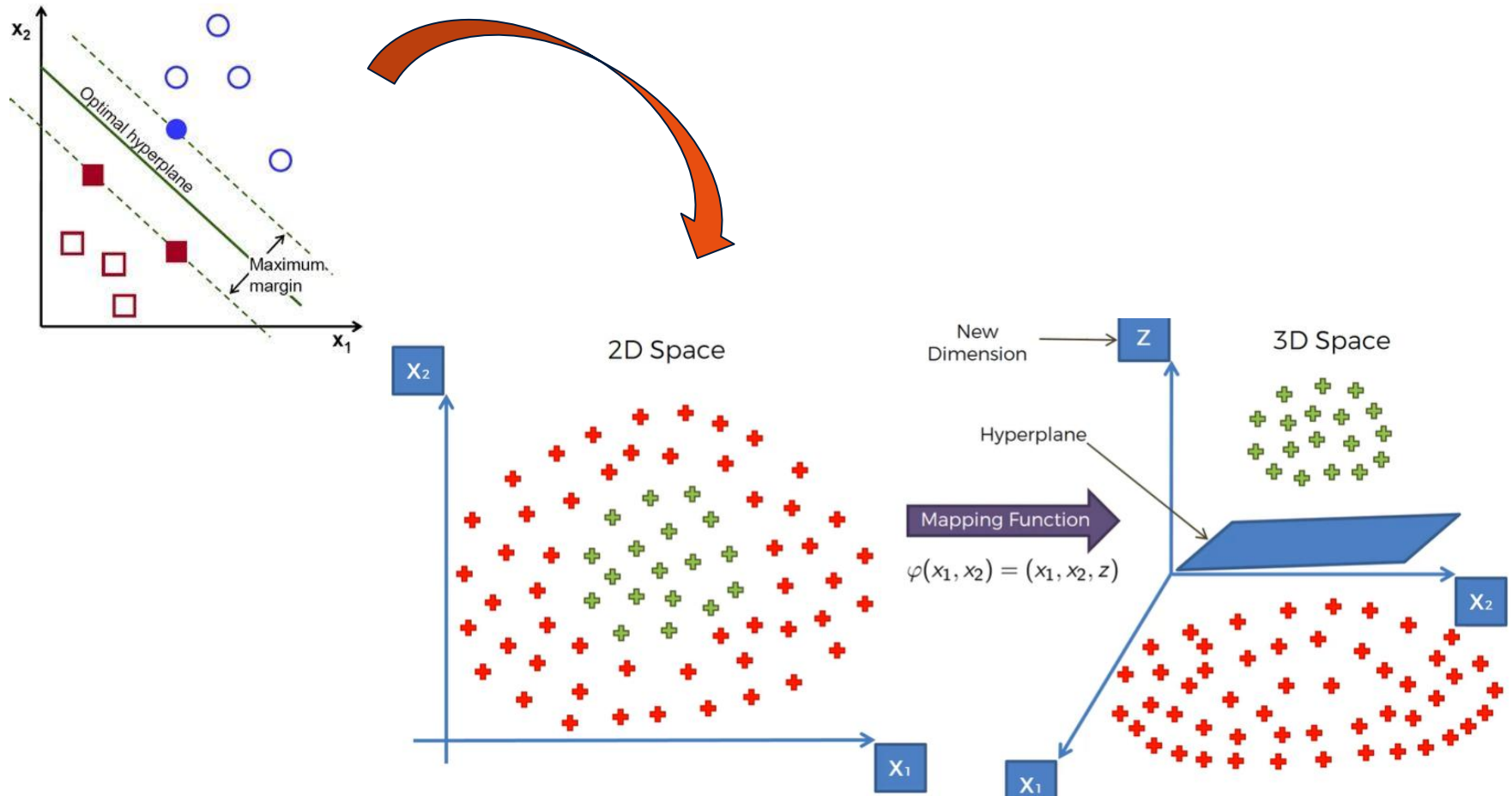
1	2	31	2	9	7	34	22	11	5
11	92	4	3	2	2	3	3	2	1
3	9	13	8	21	17	4	2	1	4
8	32	1	2	34	18	7	78	10	7
9	22	3	9	8	71	12	22	17	3
13	21	21	9	2	47	1	81	21	9
21	12	53	12	91	24	81	8	91	2
61	8	33	82	19	87	16	3	1	55
54	4	78	24	18	11	4	2	99	5
13	22	32	42	9	15	9	22	1	21

Sparse Matrix

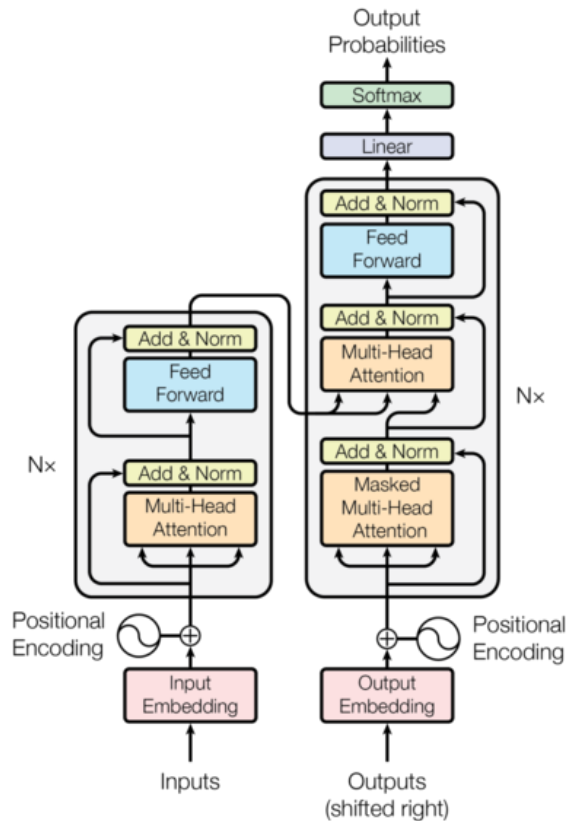
1	.	3	.	9	.	3	.	.	.
11	.	4	.	.	.	.	.	2	1
.	.	1	.	.	.	4	.	1	.
8	.	.	.	3	1	.	.	.	.
.	.	.	9	.	.	1	.	17	.
13	21	.	9	2	47	1	81	21	9
.	.	.	.	.	.	.	.	.	.
.	.	.	.	19	8	16	.	.	55
54	4	.	.	.	11	.	.	.	.
.	.	2	.	.	.	.	22	.	21

TF-IDF gives us a sparse doc-term matrix (#vocabulary x #docs). Word2Vec a very dense doc-terms matrix (300 x #docs). As a result we can no longer use a linear SVM.

From LIN-SVM to Gaussian or Kernel-SVM



How about Transformers for Text-Classification?



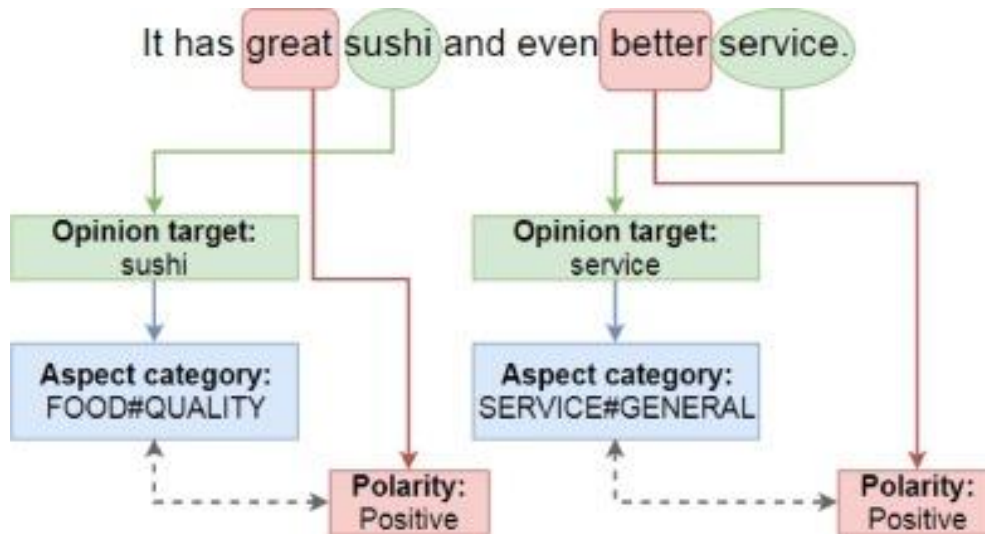
Vaswani, A., Shazeer, N.M., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., & Polosukhin, I. (2017). Attention is All you Need. NIPS.

Sentiment & Emotion Analysis

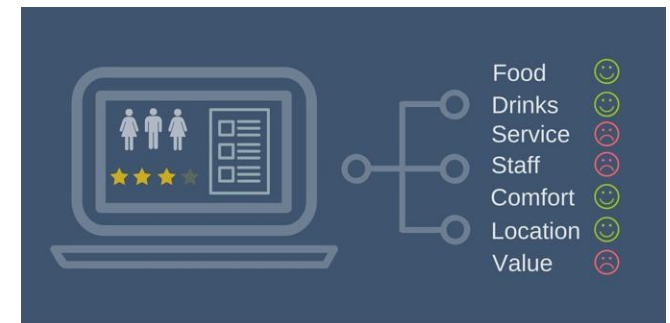


Maastricht University

Aspect or Target-Based Sentiment Mining



Analysis:



See also: Gerolemou, Z., & Scholtes, J. (2019). Target-Based Sentiment Analysis as a Sequence-Tagging Task. Paper presented at BNAIC 2019, Brussels, Belgium.

Bing Liu, Sentiment Analysis and Opinion Mining - 2012

- An *opinion* is a quintuple

$$(e_j, a_{jk}, so_{ijkl}, h_i, t_l),$$

where

- e_j is a target entity.
- a_{jk} is an aspect/feature of the entity e_j .
- so_{ijkl} is the sentiment value of the opinion from the opinion holder h_i on feature a_{jk} of entity e_j at time t_l . so_{ijkl} is +ve, -ve, or neu, or more granular ratings.
- h_i is an opinion holder.
- t_l is the time when the opinion is expressed.

- **Subjectivity** and **emotion**.
- **Sentence subjectivity**: An *objective sentence* presents some factual information, while a *subjective sentence* expresses some personal feelings, views, emotions, or beliefs.
- **Emotion**: Emotions are people's subjective feelings and thoughts.
 - No agreed set of basic emotions of people among researchers.
 - **Based on (Parrott, 2001), people have six main emotions**,
 - love, joy, surprise, anger, sadness, and fear.
 - **Strengths of opinions/sentiments are related to certain emotions**, e.g., joy, anger.
 - However, the concepts of emotions and opinions are not equivalent.

Not just ONE problem

- $(e_j, a_{jk}, so_{ijkl}, h_i, t_l)$,
 - e_j - a target entity: Named Entity Extraction (more)
 - a_{jk} - an aspect of e_j : Information Extraction
 - so_{ijkl} is sentiment: Sentiment Identification
 - h_i is an opinion holder: Information/Data Extraction
 - t_l is the time: Information/Data Extraction
 - 5 pieces of information must match
- Coreference resolution
- Synonym match (voice = sound quality)
- ...

Combining it all

- 
1. Find documents with sentiment or opinion words
 2. Find subjective sentences
 3. Identify individual opinion words / phrases / sentences.
 4. Normalize opinion words (lexical, synonym, ...)
 5. Investigate modifiers (negations, boosters, sarcasm, ...) and modify impact opinion words accordingly
 6. Compute total opinion by sentence and/or document
 7. Measure impact of publication
 8. Store opinion value
 9. Visualize (geo, time, network, ...)

What sentiment value for each word?

- Lists of Positive and Negative opinion words
- Word Lists with sentiment values (e.g., -2 to +2 where -2 is most negative and +2 most positive)
- Probabilistic or Entropy models (w = word, e = emotion)

$$P(w|e) = \frac{|(w, e)|}{\sum_{w' \in W} |(w', e)|}$$

Other sentiment modifiers to deal with

- Sentiment/opinion shifters (also called valence shifters are words and phrases that can shift or change opinion orientations.
 - Negation words like not, never, cannot, etc., are the most common type.
 - Many other words and phrases can also alter opinion orientations. E.g., modal auxiliary verbs (e.g., would, should, could, etc) “The brake could be improved.”
 - Some pre-suppositional items also can change opinions, e.g., barely and hardly”: “It hardly works.” (comparing to “it works”). It presupposes that better was expected.
 - Words like fail, omit, neglect behave similarly: “This camera fails to impress me.”
 - Sarcasm changes orientation too: “What a great car, it did not start the first day.”

Problems with keyword approaches

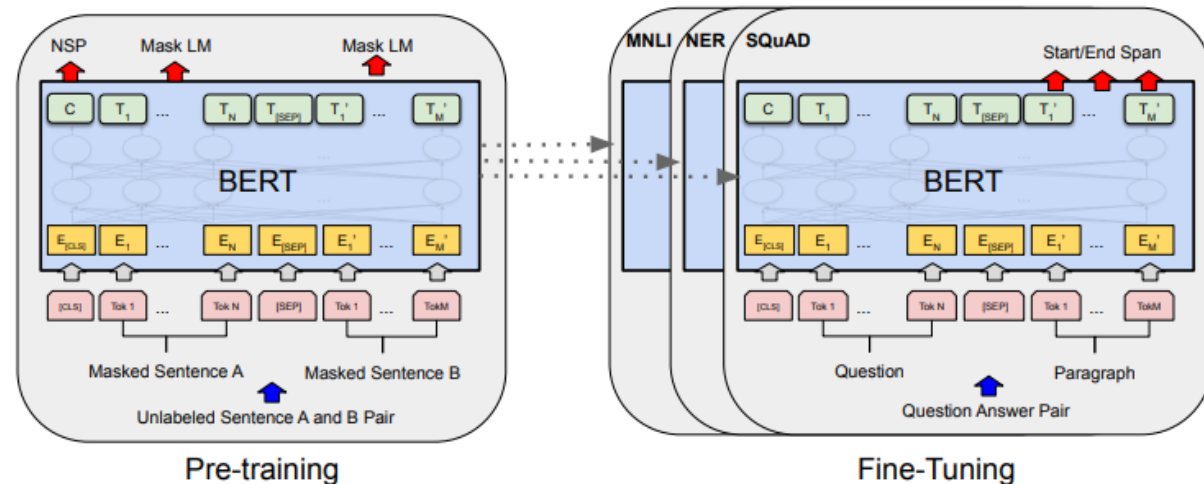
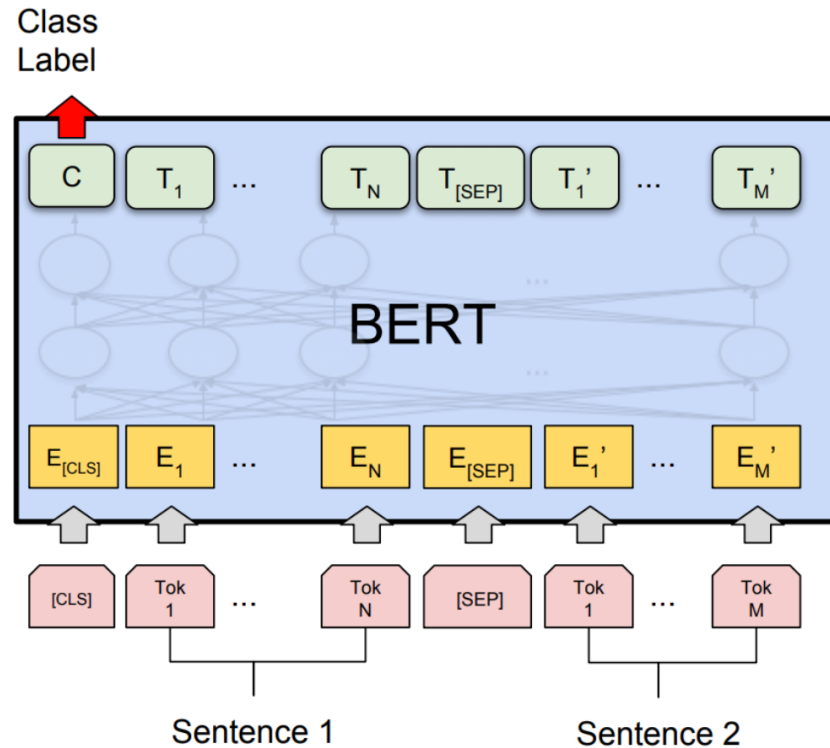
- In general, a high recall (especially with large gazetteers).
- But cannot capture linguistic ambiguity and complexity: negations, different meanings, boundaries, subjectivity/objectivity, errors, etc. So, this approach always has low precision.

How can we better incorporate understanding of these linguistic issues in our systems?



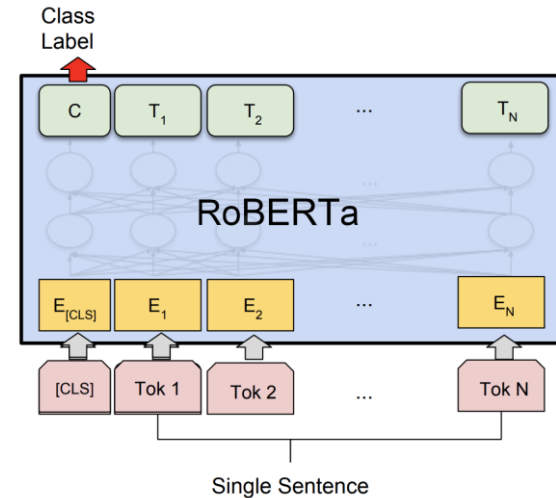
Remember BERT

- Transformer
- Encoder only
- Context sensitive embeddings (bi-directional)
- Trained on Mask Language
- Next Sentence Prediction
- Can be fine tuned for a variety of linguistic tasks such as NER and Sentiment detection.
- Can also be used for POS-tagging



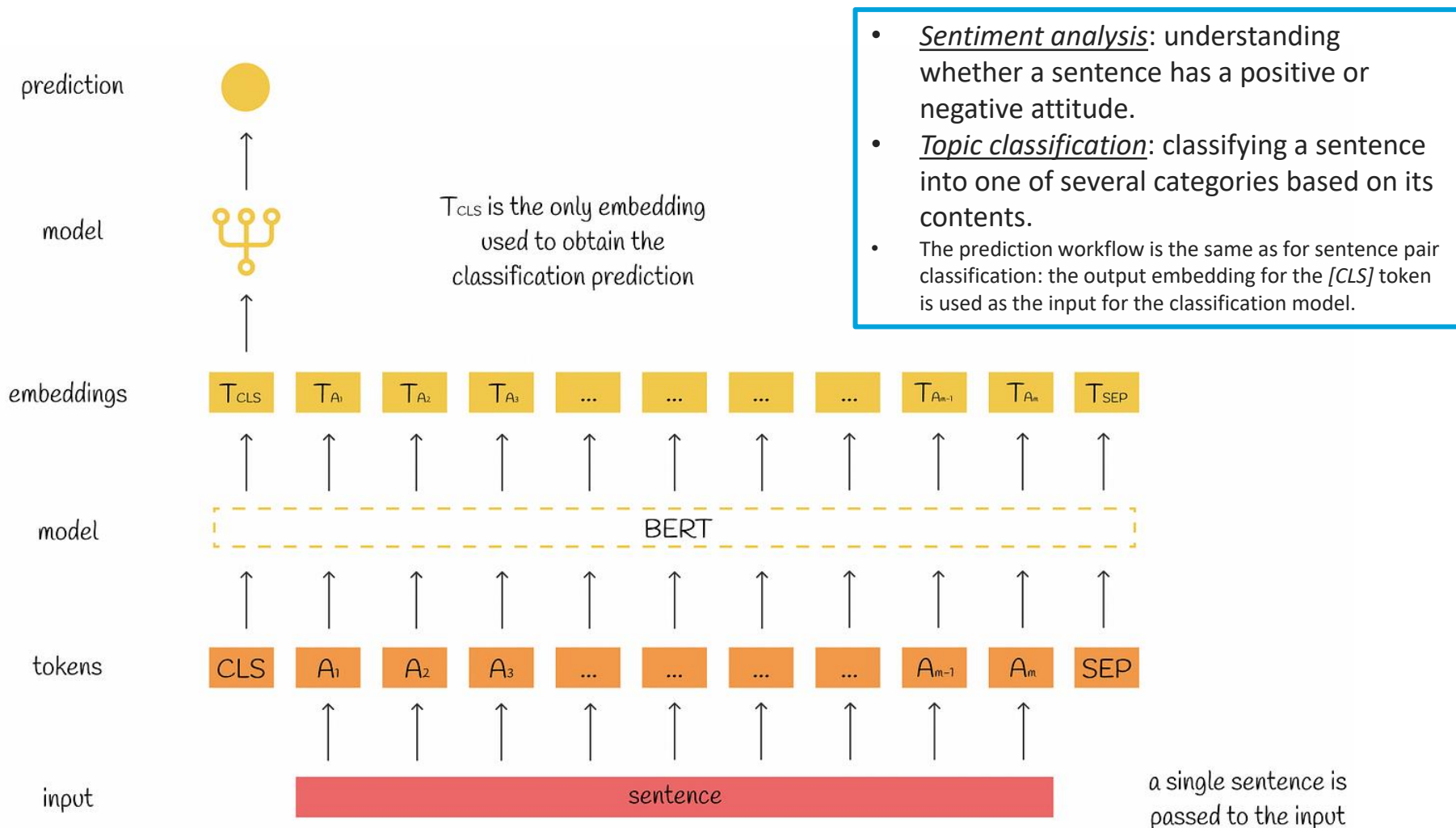
BERT for Sentiment Mining

- Take standard BERT
- Fine-tune on sentiment mining
- SST-2 data set
- We will do this in the tutorial

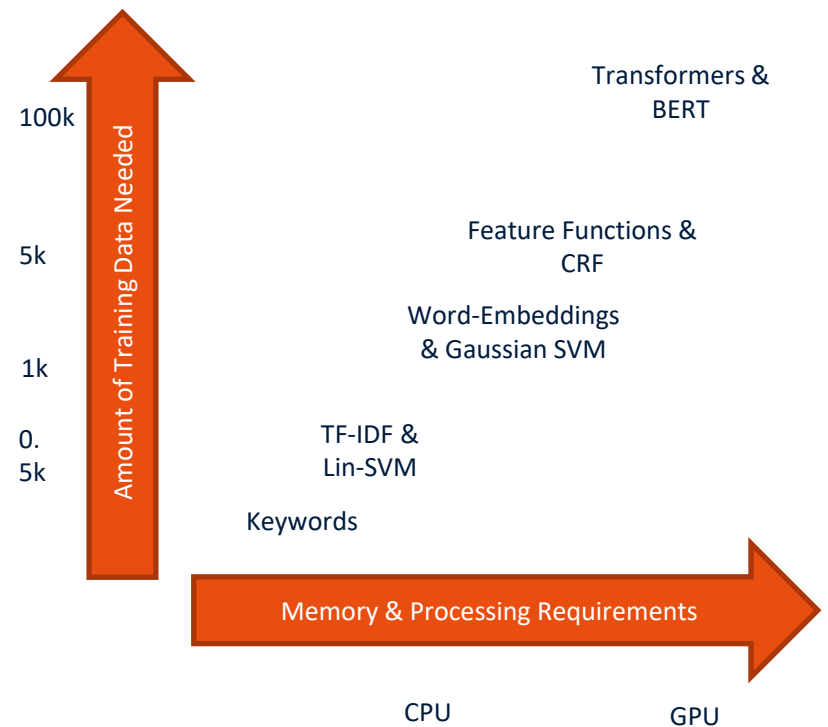
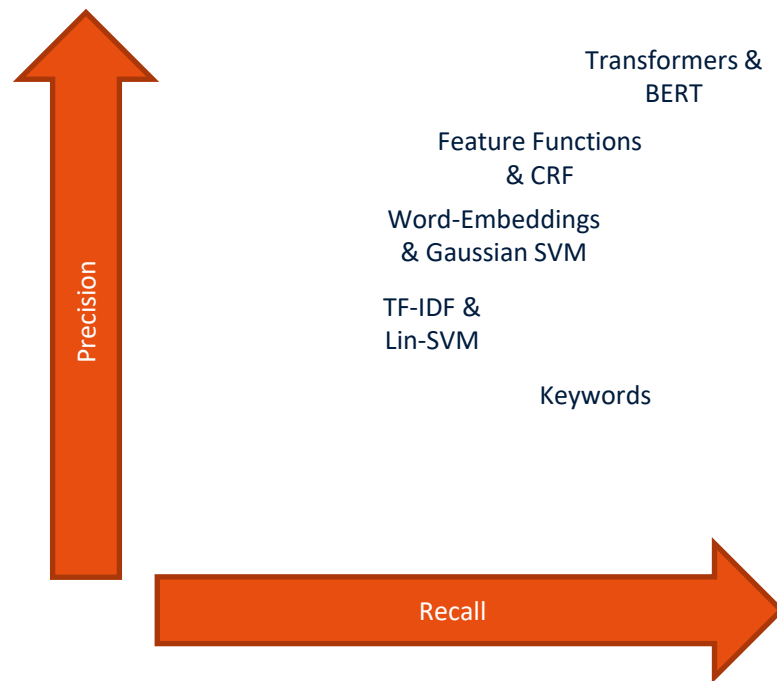


Model	SST-2(ALL)	SST-2(Root)	SST-5(ALL)	SST-5(Root)
RNN [32]	83.6	80.2	75.9	41.4
LSTM [33]	85.2	82.6	80.7	42.3
BiLSTM [34]	86.1	84.8	83.5	45.6
CNN [35]	85.7	85.1	84.2	46.5
CNN-LSTM [36]	86.2	85.7	83.9	47.4
BERT-base	92.7	90.9	81.3	50.4
DaBERT	94.8	92.6	85.3	53.9

BERT: Single Sentence Classification



Quality and Training Data / Computation Requirements

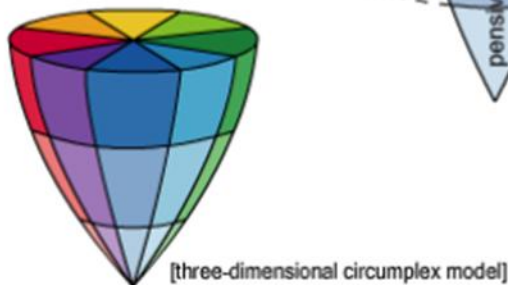
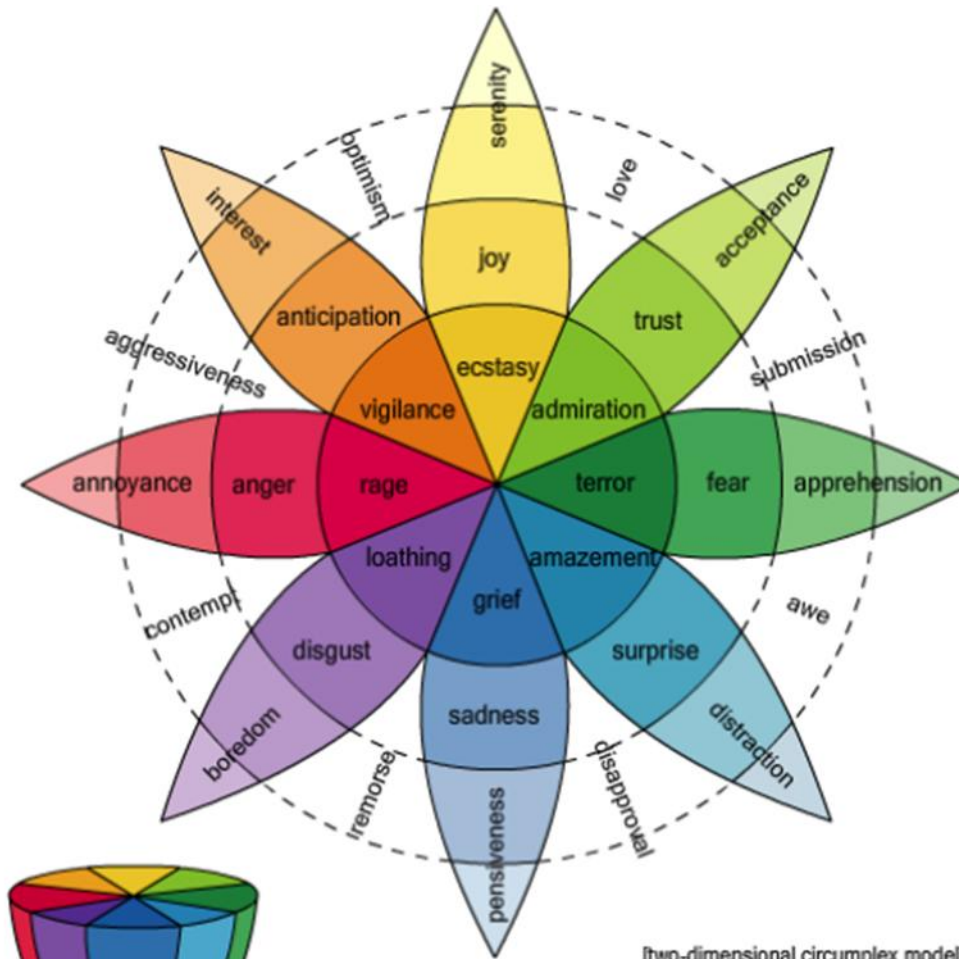


From Sentiment Mining to Emotion Mining

- Emotions are mental states accompanied by psychological changes.
- Emotions can transfer into other emotions.
- Basic emotions can be combined into more complex emotions.
- Emotions are partly expressed by language, and partly by several forms of “ body language”
- Research community does not agree on *one and only* model to represent emotions, emotional state and relevant features (linguistic, physical, previous state, relational, gender, culture, mental state, ...) to determine and measure emotions.



- Sentiments are positive or negative opinions.
- Opinions can be measured by measuring only textual expressions without other necessary input.



Expansion of Ekman's 6 emotions with 2 additional: trust & anticipation.

The closer to the center, the more intense.

8 basic emotions form 4 contracting pairs of emotions:

- joy&sadness;
- anger&fear;
- trust&disgust;
- anticipation&surprise

[Plutchik, 1980 & 1992]

Emotion Modifiers

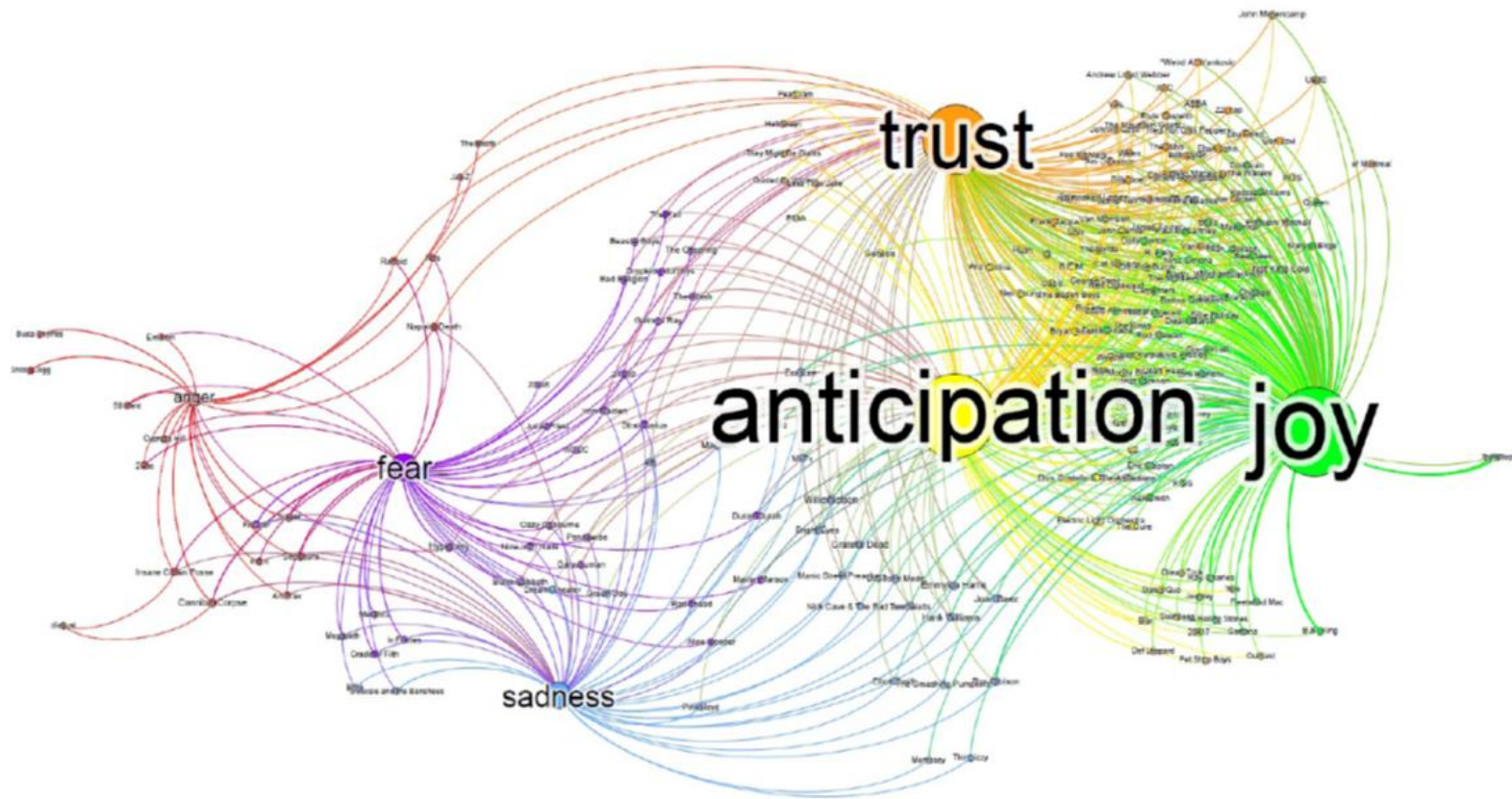
- Same as with sentiments, but with emotions there are more dimensions, and it is more complex.
- Also take emotion modifications and transfer in consideration.
- Basic emotions can be combined into more complex emotions.
- Really need Dependency Grammars
- Don't forget about:

Emoticons

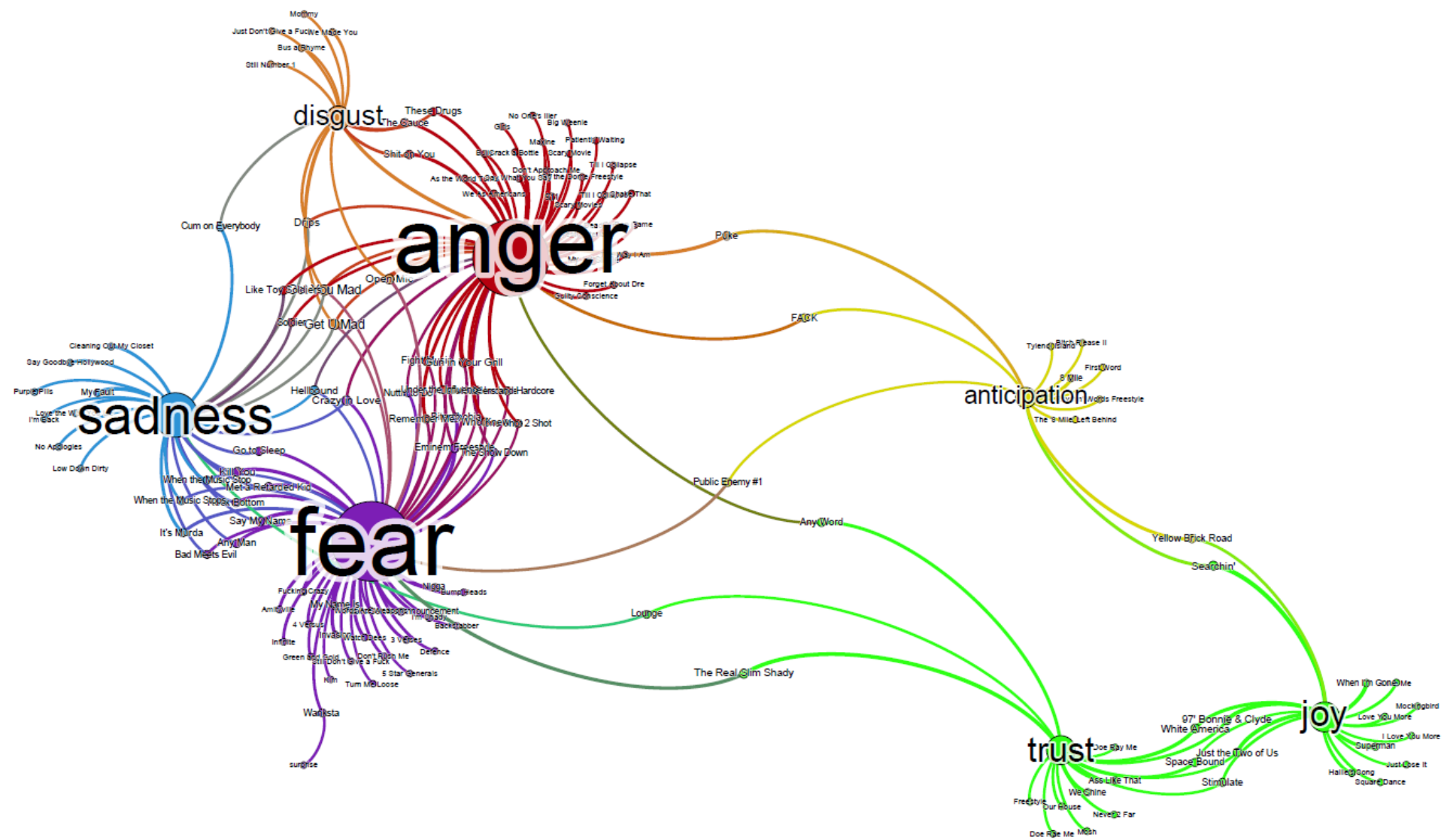


Here too, BERT is the SOTA. Why?

Emotion mining on 220.000 song lyrics



Torsten Schuster and Alexey Sibirtsev, IRTM Practical Assignment 2014, Information Retrieval and Text Mining in Lyrics





Sentiment and Emotion Mining Frequent Error Sources

- Wrong or no negation handling
- No objectivity / subjectivity filtering

Be Aware: “Negation handling with Deep Learning does not always work ...”

Context	BERT _{LARGE} predictions
<i>A robin is a ____</i>	<i>bird, robin, person, hunter, pigeon</i>
<i>A daisy is a ____</i>	<i>daisy, rose, flower, berry, tree</i>
<i>A hammer is a ____</i>	<i>hammer, tool, weapon, nail, device</i>
<i>A hammer is an ____</i>	<i>object, instrument, axe, implement, explosive</i>
<i>A robin is not a ____</i>	<i>robin, bird, penguin, man, fly</i>
<i>A daisy is not a ____</i>	<i>daisy, rose, flower, lily, cherry</i>
<i>A hammer is not a ____</i>	<i>hammer, weapon, tool, gun, rock</i>
<i>A hammer is not an ____</i>	<i>object, instrument, axe, animal, artifact</i>

Table 13: NEG-88-SIMP predictions by BERT_{LARGE}

Source: What BERT is not: Lessons from a new suite of psycholinguistic diagnostics for language models.
Allyson Ettinger, July 2019. <https://arxiv.org/abs/1907.13528>

Objectivity / Subjectivity Detection



I love this movie!

There is fighting,
killing and blood all
over the script

Toxic Text Detection



Maastricht University

Categories of Toxicity

Detection models are generally trained to classify text into several sub-types to allow for nuanced moderation:

- **Insult:** Rude or disrespectful comments aimed at an individual or group.
 - Example: "You are absolutely useless and should never have been hired."
- **Hate Speech:** Content that attacks, threatens, or degrades individuals or groups based on protected attributes.
- **Threat or Pressure:** Statements of intent to inflict harm or violence.
 - Example: "If I ever see you in person, you'll regret ever posting that."
- **Obscenity:** Use of profanity or vulgar language that violates community standards.
- **Severe Toxicity:** Highly offensive or hateful content that typically requires immediate automated removal or escalation.



The “Context” Challenge

Simple keyword lists fail. BERT-based models are preferred for high precision, because:

- **Keyword Failure** (False Positives): A script for a movie like Pulp Fiction contains significant "violence and blood," but it is an artistic expression, not a personal attack.
- **Sarcasm & Sentiment Shifters**: Words that are usually positive can become toxic when used sarcastically.
 - Example: "Oh, what a brilliant idea. I'm sure your IQ is just off the charts."
(Detected as an insult through contextual embeddings rather than raw keywords).

Comparison

Method	Strength	Example Implementation
Keywords/Gazetteers	High Recall	Flagging any message containing a "blacklisted" word
Lin-SVM with TF-IDF	Efficiency	Fast baseline for large-scale filtering of obvious spam/toxicity
BERT / RoBERTa	High Precision	fine-tuning the [CLS] token output to classify the overall toxicity of a sentence pair or single sentence

In agentic workflows, toxic text detection acts as a Guardrail. Before an AI Agent processes a user's request or generates a response, a classification layer checks the input to ensure it meets safety requirements, preventing the system from engaging with or generating harmful content.

If one has a collection of sentences that express the toxicity that needs to be detected, then all the above methods can be setup using that data set.

Questions?